



РОСНЕФТЬ

**ВНУТРЕННЕЕ УСТРОЙСТВО
СБОРКИ МУСОРА В
СРУТНОН 3.14+**

**МИРЯНОВ СЕРГЕЙ
PH-ТЕХНОЛОГИИ**

#ITGORKY 2025

- Старший эксперт в **РН-ТЕХНОЛОГИИ**
- Мы занимаемся разработкой наукоемкого десктопного ПО
- Для нефтянки и не только

<https://github.com/python/cpython>

```
3.15 @ c5180d74e199e5bdb8f1c149dc3e956071ebf365  
3.14 @ 80e59a8d7e4c9506124e3457030efc20cb499423  
3.13 @ d8d11d50bf54df3005c4833282a1988fe9b4a0e5
```

БАЗА

- Автоматическое управление памятью
- Подсчет ссылок
- Циклические ссылки

СБОРЩИК МУСОРА С ПОКОЛЕНИЯМИ

```
def process_file(filename: str) -> float:
    levels = []
    with open_video(filename) as f:
        for video_frame in f.get_frames():
            level = video_frame.get_violence_level()
            levels.append(level)

    return np.median(levels)

def process_files(levels: {str, float}, directory: str):
    for _, dirs, files in os.walk(directory):
        for file in files:
            level = process_file(file)
            levels[file] = level

        for dir in dirs:
            process_files(levels, dir)

violence_level = {}
process_files(violence_level, 'd:/video/anime')
```

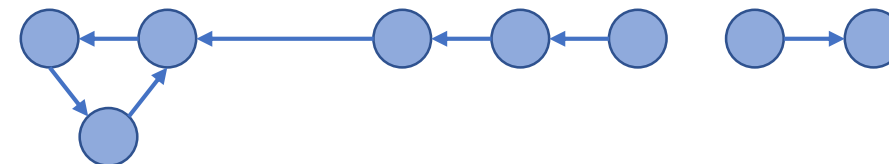
```
def process_file(filename: str) -> float:
    levels = []
    with open_video(filename) as f:
        for video_frame in f.get_frames():
            level = video_frame.get_violence_level()
            levels.append(level)

    return np.median(levels)

def process_files(levels: {str, float}, directory: str):
    for _, dirs, files in os.walk(directory):
        for file in files:
            level = process_file(file)
            levels[file] = level

        for dir in dirs:
            process_files(levels, dir)

violence_level = {}
process_files(violence_level, 'd:/video/anime')
```



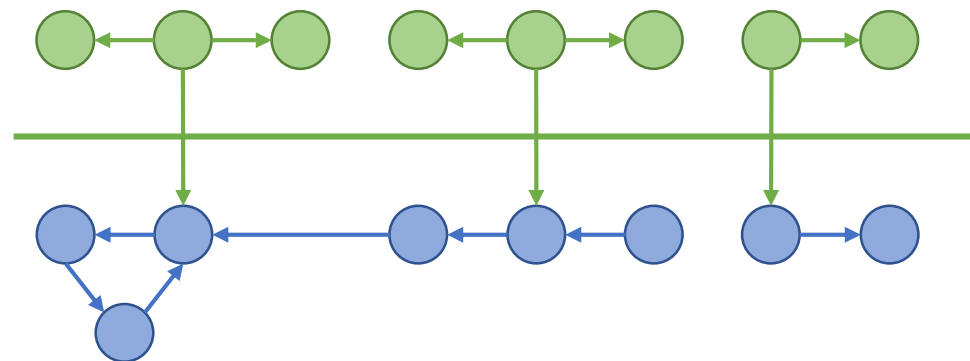

```
def process_file(filename: str) -> float:
    levels = []
    with open_video(filename) as f:
        for video_frame in f.get_frames():
            level = video_frame.get_violence_level()
            levels.append(level)

    return np.median(levels)

def process_files(levels: {str, float}, directory: str):
    for _, dirs, files in os.walk(directory):
        for file in files:
            level = process_file(file)
            levels[file] = level

        for dir in dirs:
            process_files(levels, dir)

violence_level = {}
process_files(violence_level, 'd:/video/anime')
```



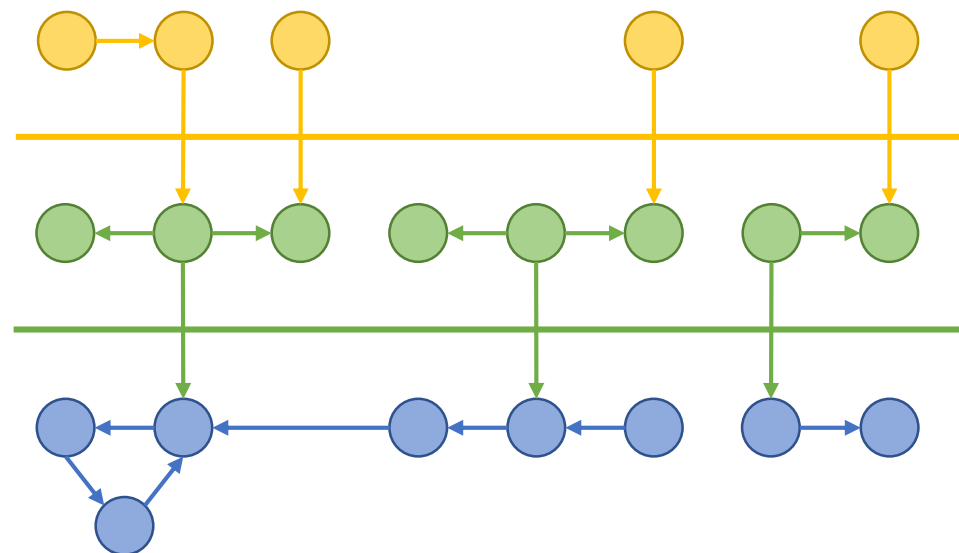
```
def process_file(filename: str) -> float:
    levels = []
    with open_video(filename) as f:
        for video_frame in f.get_frames():
            level = video_frame.get_violence_level()
            levels.append(level)

    return np.median(levels)

def process_files(levels: {str, float}, directory: str):
    for _, dirs, files in os.walk(directory):
        for file in files:
            level = process_file(file)
            levels[file] = level

        for dir in dirs:
            process_files(levels, dir)

violence_level = {}
process_files(violence_level, 'd:/video/anime')
```



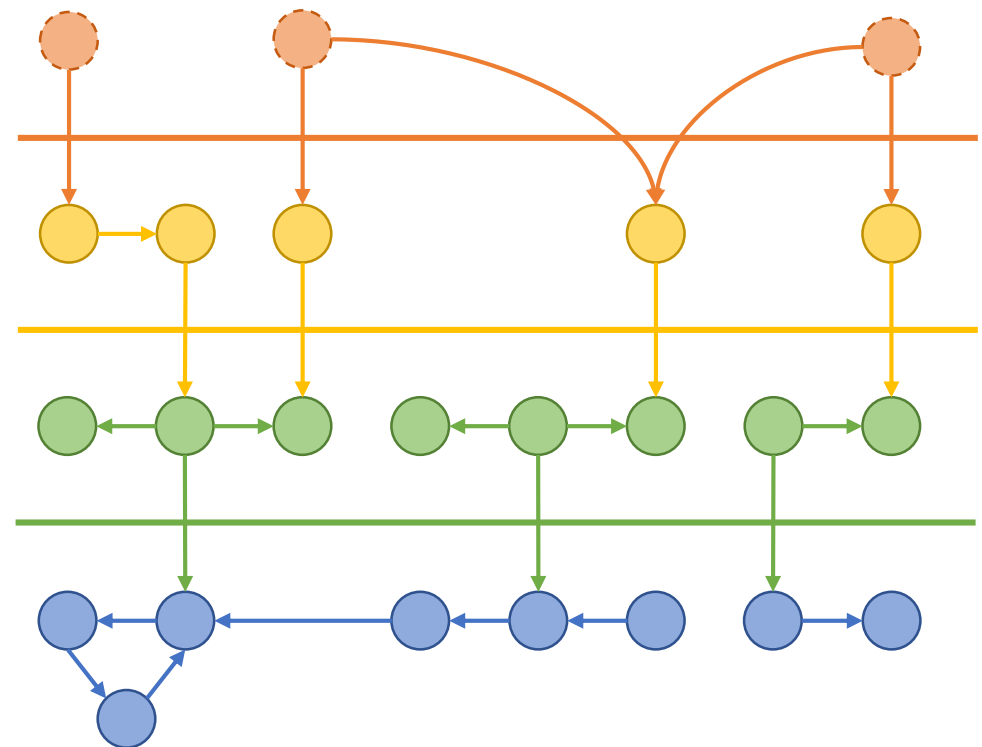
```
def process_file(filename: str) -> float:
    levels = []
    with open_video(filename) as f:
        for video_frame in f.get_frames():
            level = video_frame.get_violence_level()
            levels.append(level)

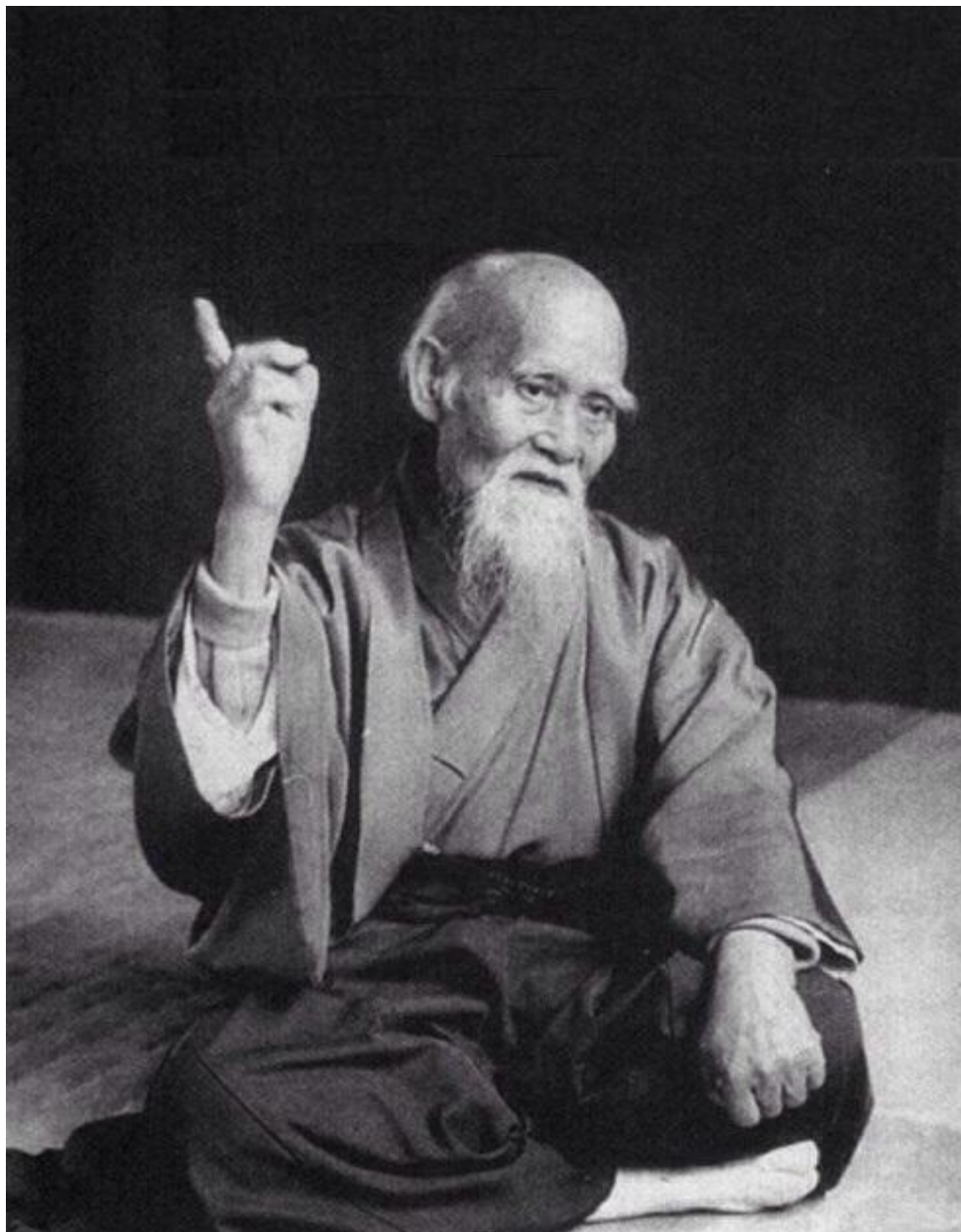
    return np.median(levels)

def process_files(levels: {str, float}, directory: str):
    for _, dirs, files in os.walk(directory):
        for file in files:
            level = process_file(file)
            levels[file] = level

        for dir in dirs:
            process_files(levels, dir)

violence_level = {}
process_files(violence_level, 'd:/video/anime')
```





*Measurements of object lifetimes proved that **young objects die young and old objects continue to live.***

```
gc.get_threshold() == (2000, 10, 10)
```

Объектов в молодом поколении

Количество сборок молодого поколения

Количество сборок среднего поколения

$long_lived_pending / long_lived_total > 25\%$

Чем больше объектов, тем реже мы запускаем полную сборку

Чем реже мы запускаем сборку, тем дольше она выполняется

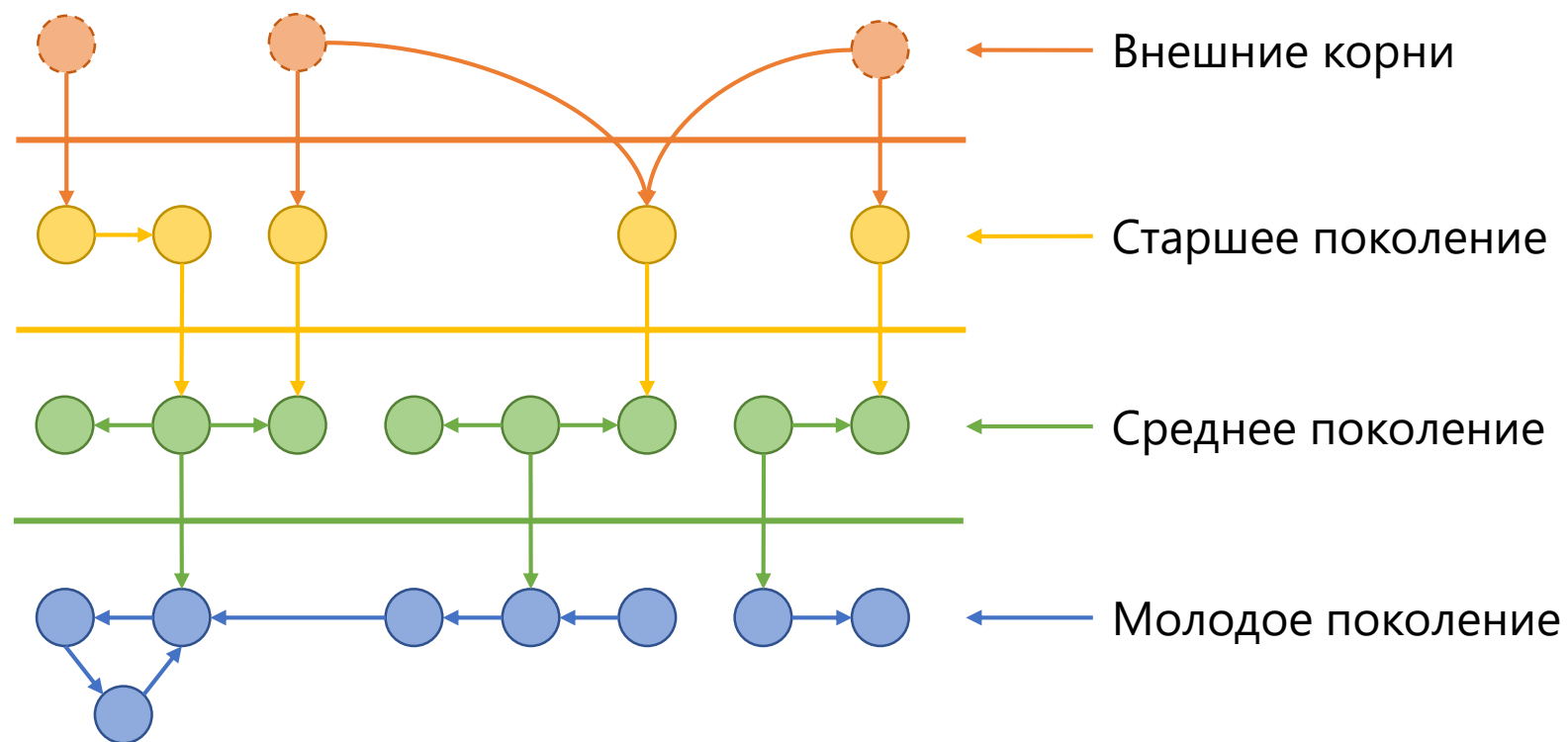
3.14+

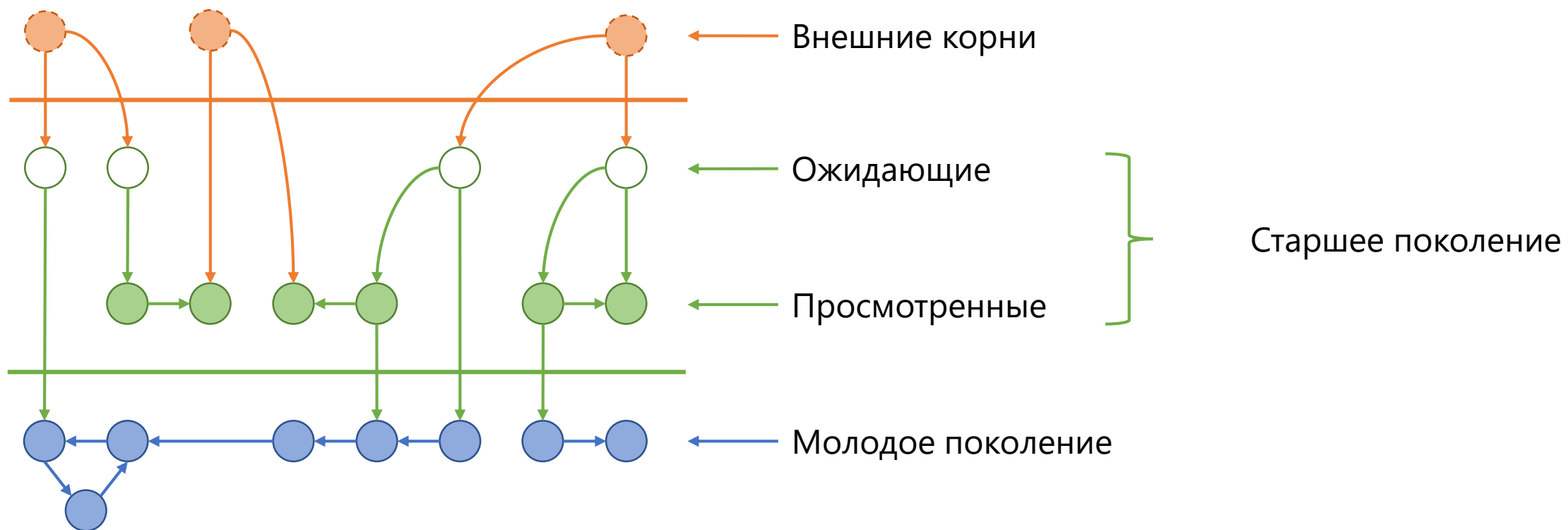
```
gc.get_threshold() == (2000, 10, 0)
```

Объектов в молодом поколении

Доля старшего поколения для сборки

Divide by 10, so that the default incremental threshold of 10 scans objects at 1% of the heap size

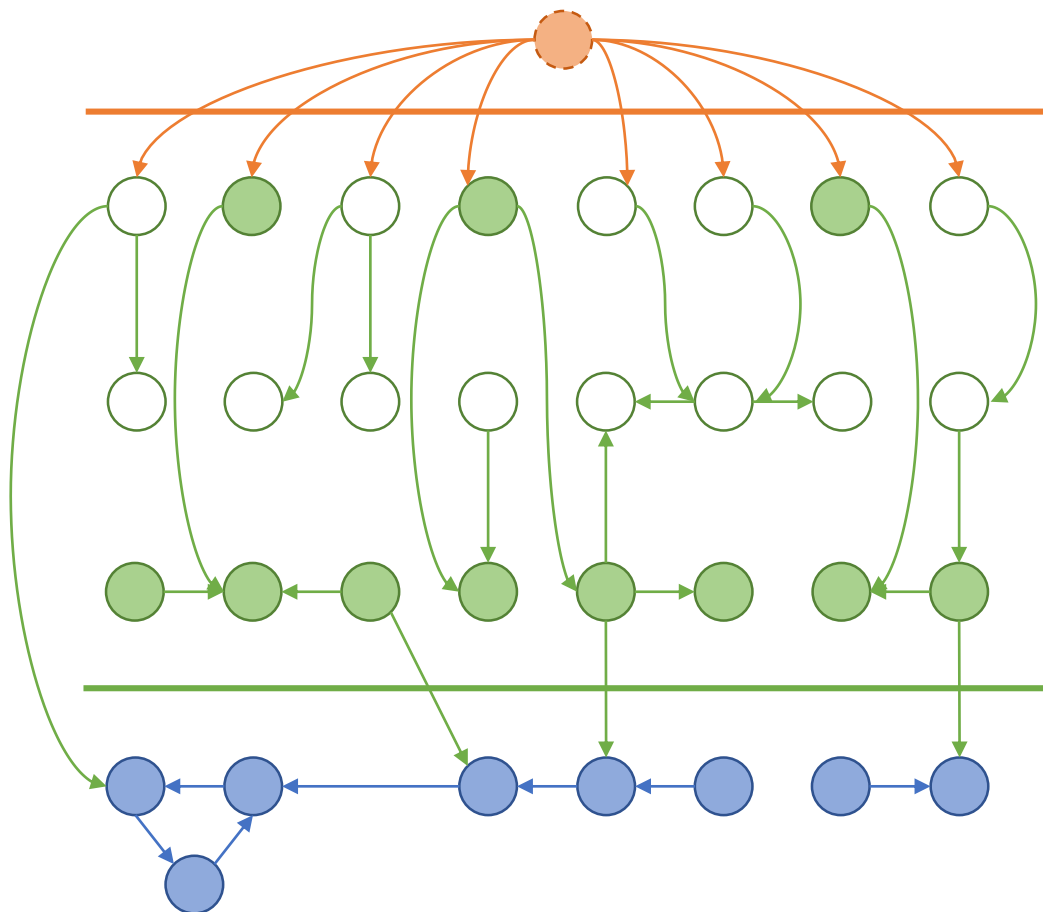




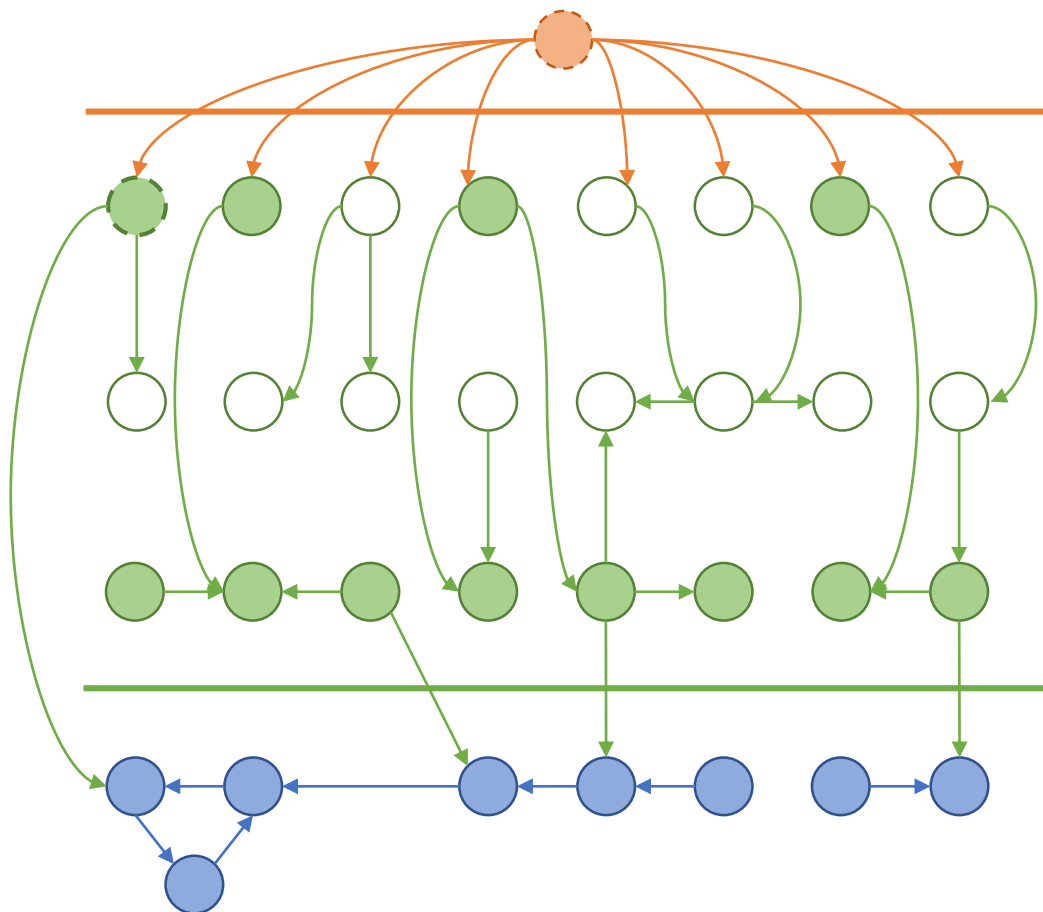
ФАЗА МАРКИРОВКИ

```
static intptr_t
mark_at_start(PyThreadState *tstate)
{
    // TO DO -- Make this incremental
    GCState *gcstate = &tstate->interp->gc;
    PyGC_Head *visited = &gcstate->old[gcstate->visited_space].head;
    Py_ssize_t objects_marked = mark_global_roots(tstate->interp, visited, gcstate->visited_space);
    objects_marked += mark_stacks(tstate->interp, visited, gcstate->visited_space, true);
    gcstate->work_to_do -= objects_marked;
    gcstate->phase = GC_PHASE_COLLECT;
    validate_spaces(gcstate);
    return objects_marked;
}
```

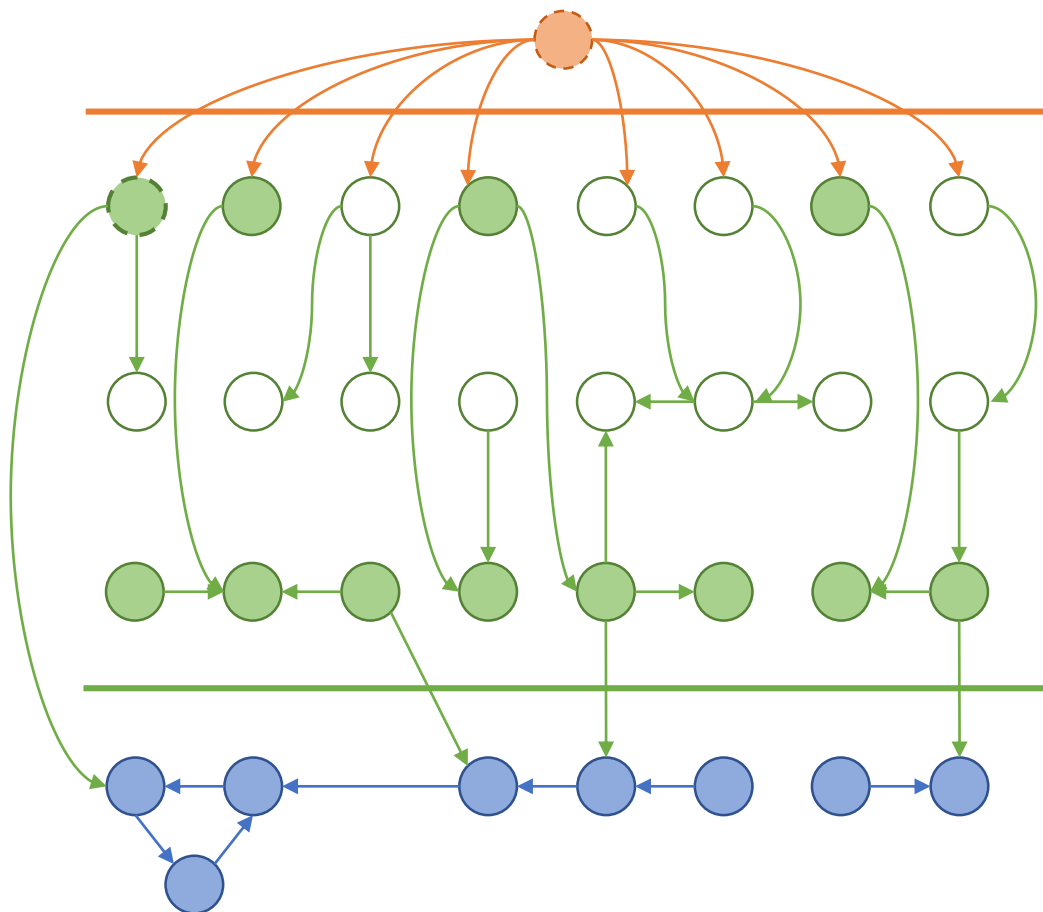
```
static intptr_t
mark_global_roots(PyInterpreterState *interp, PyGC_Head *visited, int visited_space)
{
    PyGC_Head reachable;
    gc_list_init(&reachable);
    Py_ssize_t objects_marked = 0;
    objects_marked += move_to_reachable(interp->sysdict, &reachable, visited_space);
    objects_marked += move_to_reachable(interp->builtins, &reachable, visited_space);
    objects_marked += move_to_reachable(interp->dict, &reachable, visited_space);
    // ...
    objects_marked += mark_all_reachable(&reachable, visited, visited_space);
    return objects_marked;
}
```



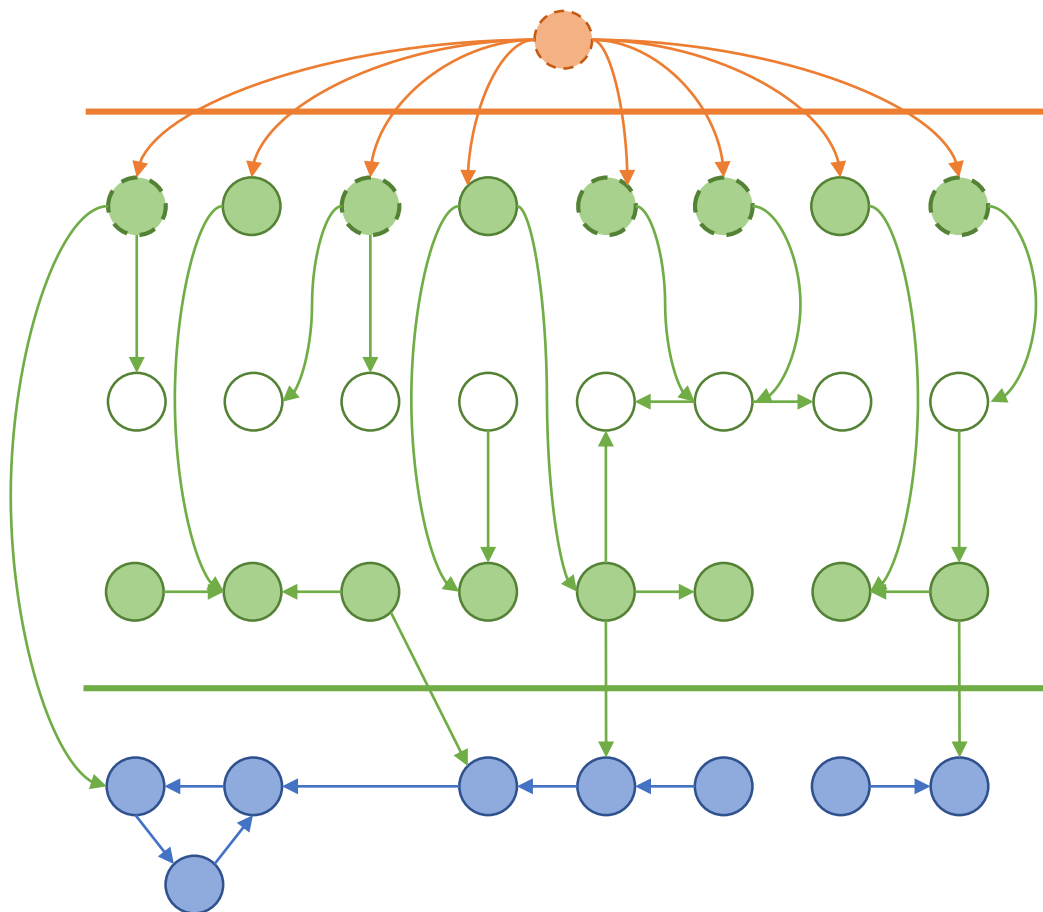
```
static intptr_t
move_to_reachable(PyObject *op, PyGC_Head *reachable, int
visited_space)
{
    // ...
    if (_PyObject_GC_IS_TRACKED(op) &&
        gc_old_space(gc) != visited_space) {
        gc_flip_old_space(gc);
        gc_list_move(gc, reachable);
        return 1;
    }
    // ...
}
```



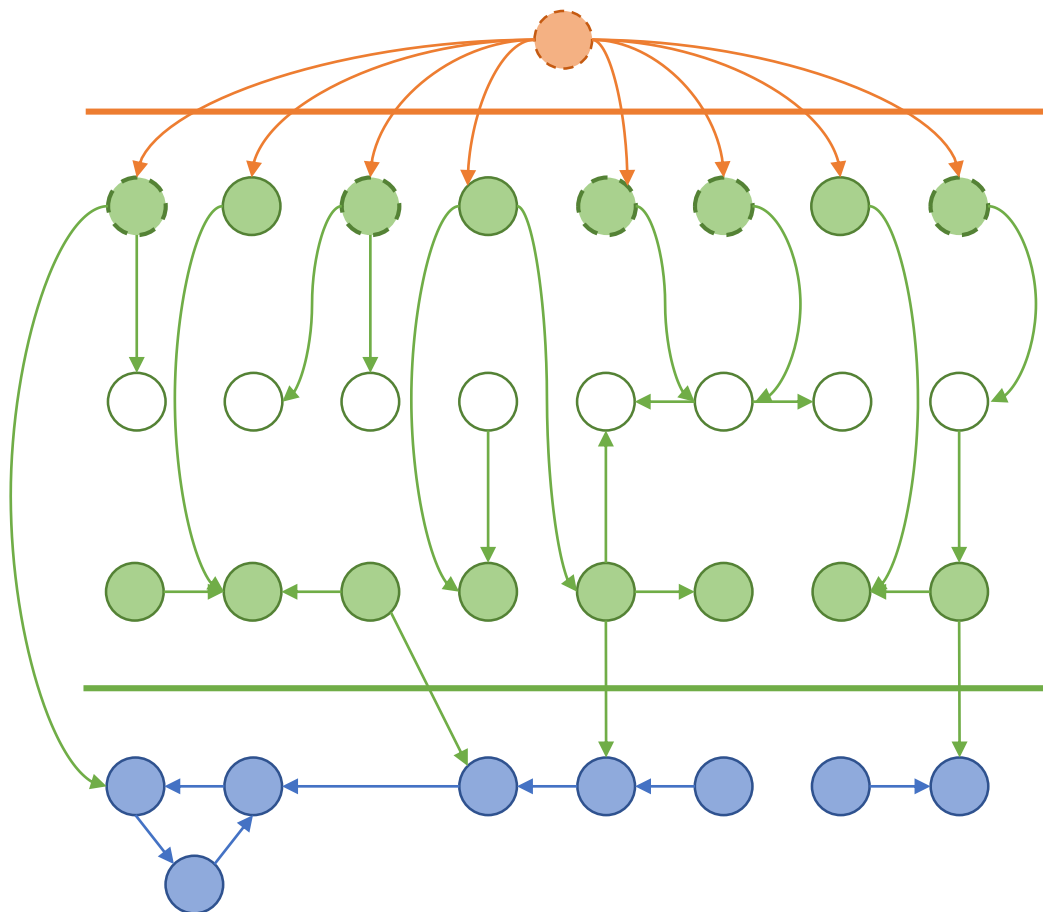
```
static intptr_t
move_to_reachable(PyObject *op, PyGC_Head *reachable, int
visited_space)
{
    // ...
    if (_PyObject_GC_IS_TRACKED(op) &&
        gc_old_space(gc) != visited_space) {
        gc_flip_old_space(gc);
        gc_list_move(gc, reachable);
        return 1;
    }
    // ...
}
```



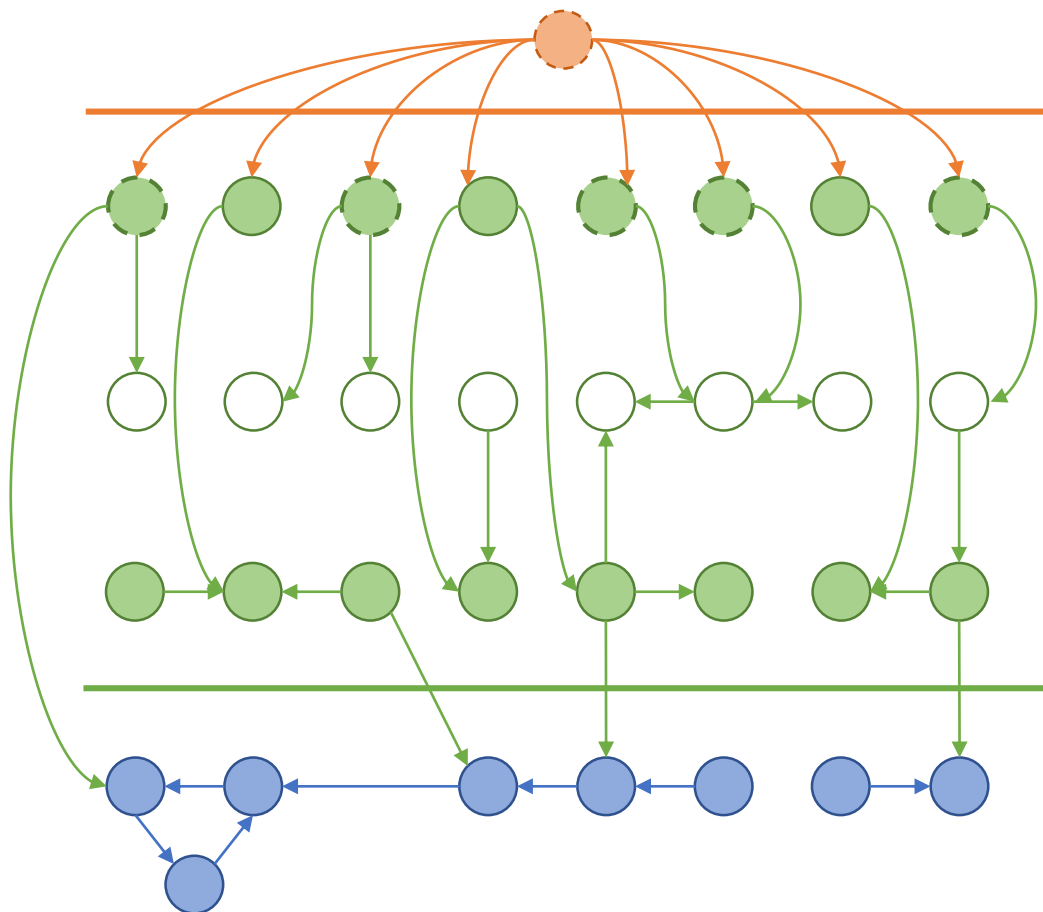
```
static intptr_t
move_to_reachable(PyObject *op, PyGC_Head *reachable, int
visited_space)
{
    // ...
    if (_PyObject_GC_IS_TRACKED(op) &&
        gc_old_space(gc) != visited_space) {
        gc_flip_old_space(gc);
        gc_list_move(gc, reachable);
        return 1;
    }
    // ...
}
```



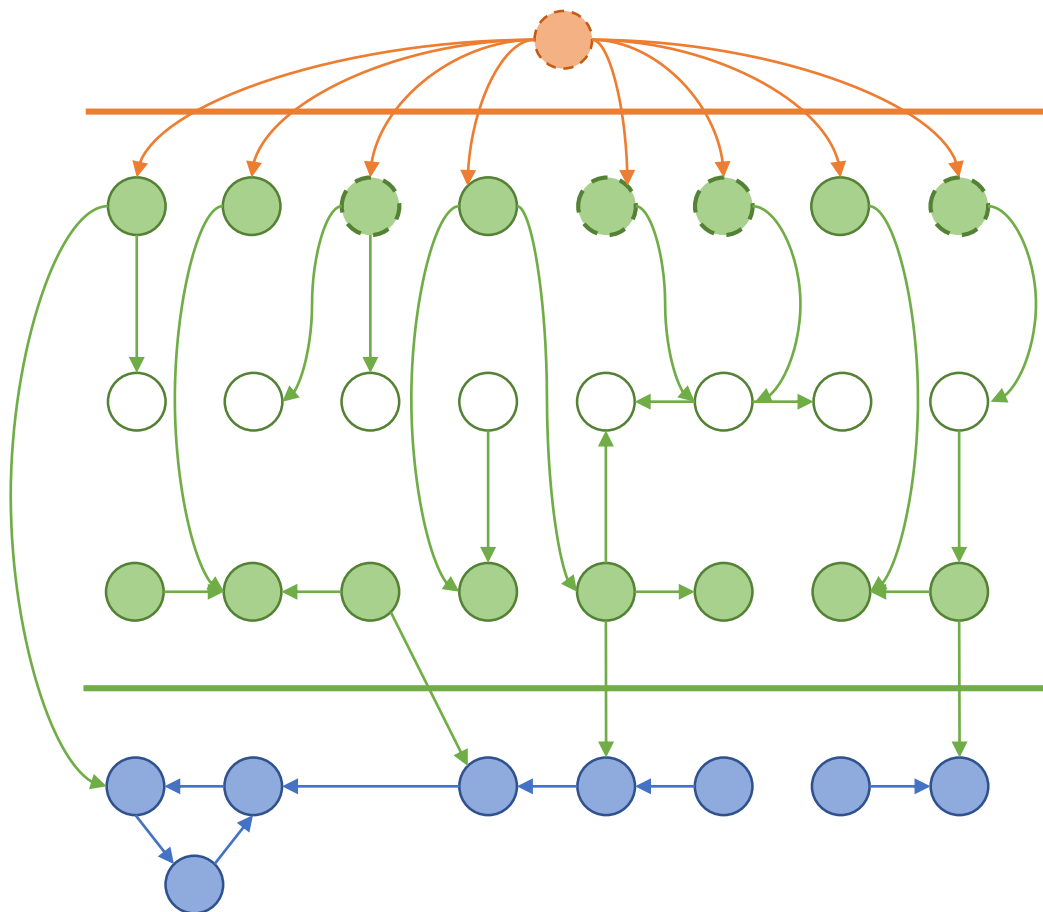
```
static intptr_t
move_to_reachable(PyObject *op, PyGC_Head *reachable, int
visited_space)
{
    // ...
    if (_PyObject_GC_IS_TRACKED(op) &&
        gc_old_space(gc) != visited_space) {
        gc_flip_old_space(gc);
        gc_list_move(gc, reachable);
        return 1;
    }
    // ...
}
```

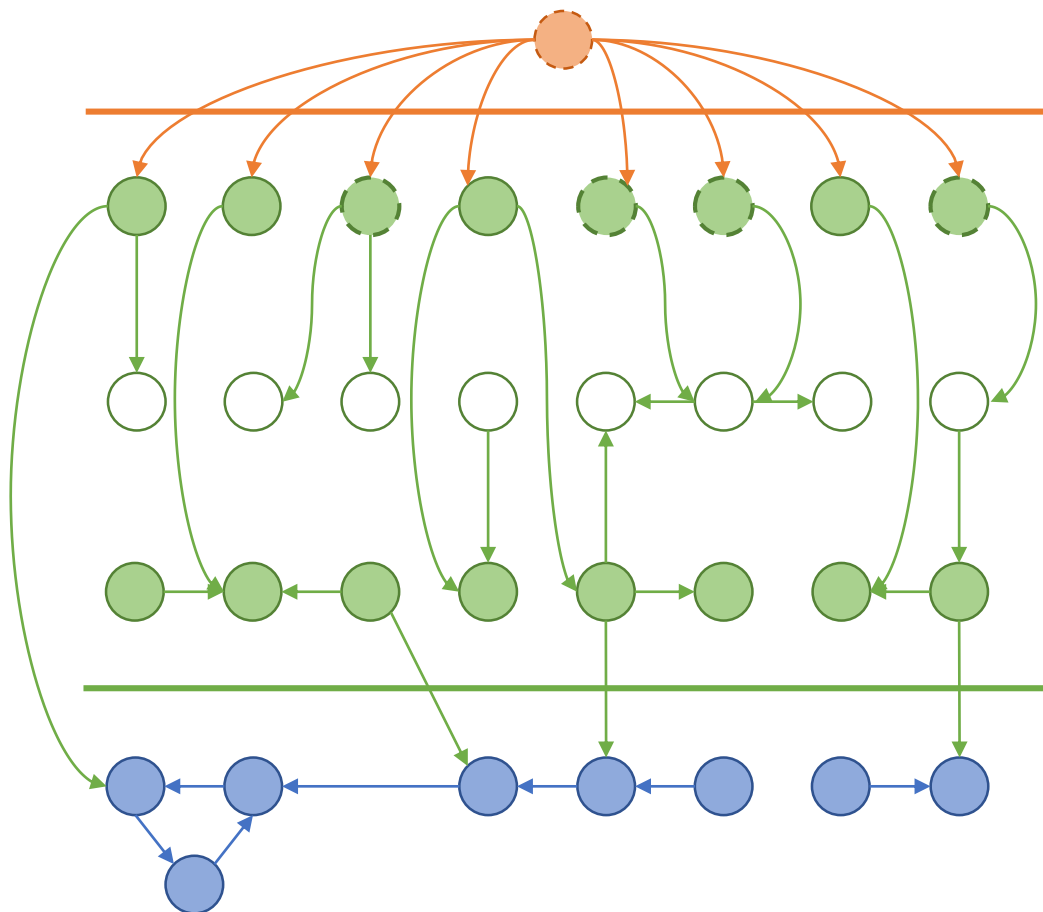
```
static intptr_t
mark_all_reachable(PyGC_Head *reachable, PyGC_Head
*visited, int visited_space)
{
    struct container_and_flag arg = {
        .container = reachable,
        .visited_space = visited_space,
        .size = 0
    };
    while (!gc_list_is_empty(reachable)) {
        PyGC_Head *gc = _PyGCHead_NEXT(reachable);
        assert(gc_old_space(gc) == visited_space);
        gc_list_move(gc, visited);
        PyObject *op = FROM_GC(gc);
        traverseproc traverse = Py_TYPE(op)->tp_traverse;
        (void) traverse(op, visit_add_to_container, &arg);
    }
    gc_list_validate_space(visited, visited_space);
    return arg.size;
}
```



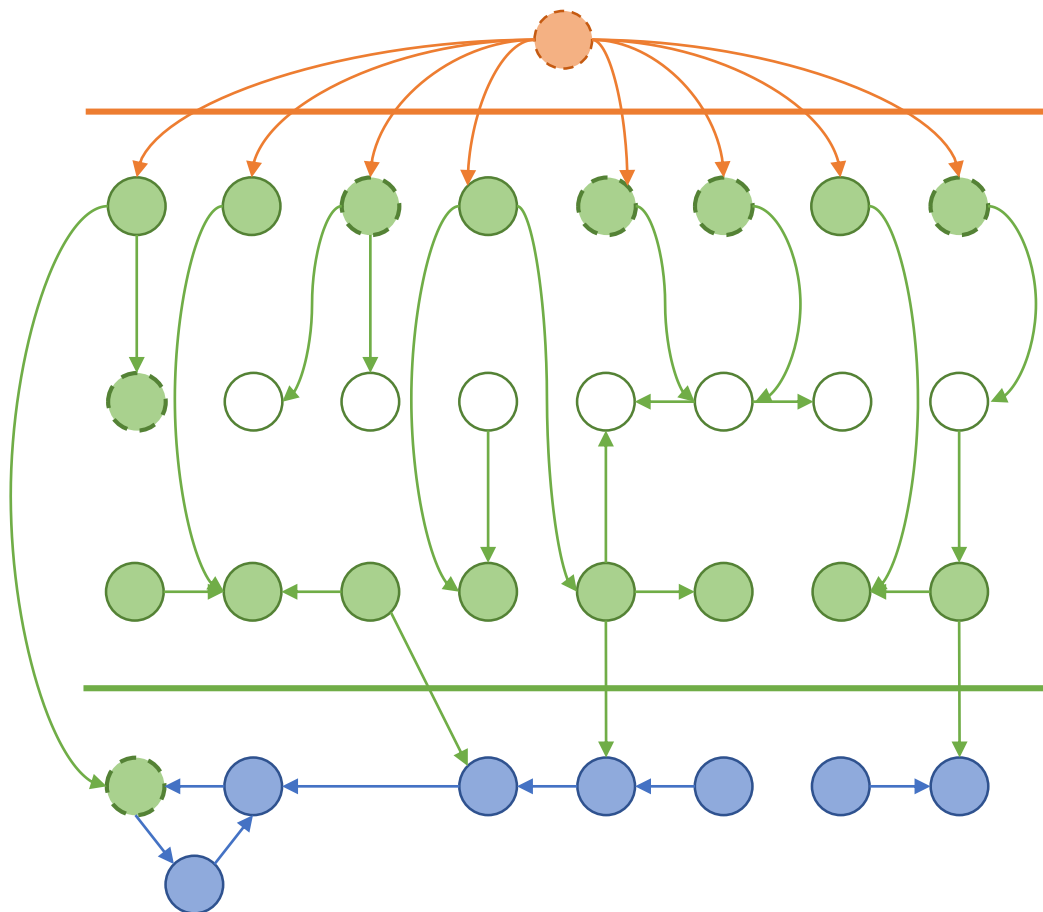
```
static intptr_t
mark_all_reachable(PyGC_Head *reachable, PyGC_Head
*visited, int visited_space)
{
    struct container_and_flag arg = {
        .container = reachable,
        .visited_space = visited_space,
        .size = 0
    };
    while (!gc_list_is_empty(reachable)) {
        PyGC_Head *gc = _PyGCHead_NEXT(reachable);
        assert(gc_old_space(gc) == visited_space);
        gc_list_move(gc, visited);
        PyObject *op = FROM_GC(gc);
        traverseproc traverse = Py_TYPE(op)->tp_traverse;
        (void) traverse(op, visit_add_to_container, &arg);
    }
    gc_list_validate_space(visited, visited_space);
    return arg.size;
}
```



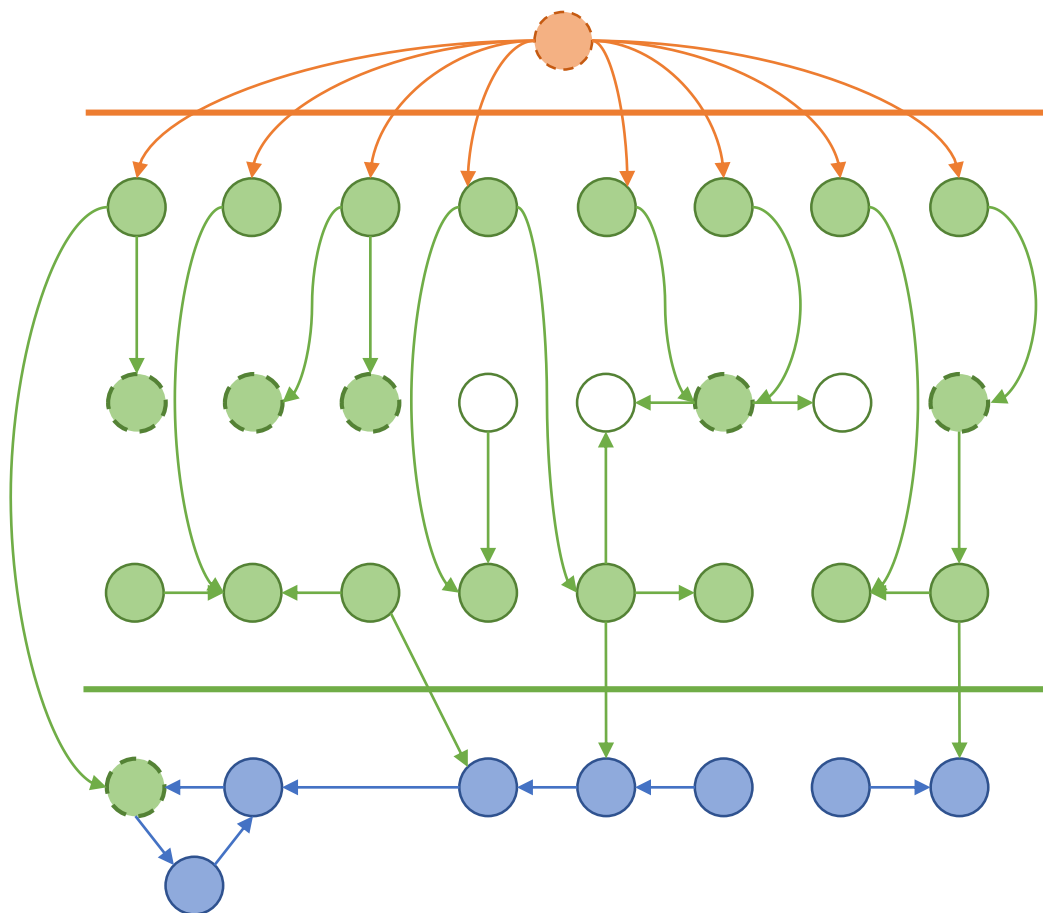
```
static intptr_t
mark_all_reachable(PyGC_Head *reachable, PyGC_Head
*visited, int visited_space)
{
    struct container_and_flag arg = {
        .container = reachable,
        .visited_space = visited_space,
        .size = 0
    };
    while (!gc_list_is_empty(reachable)) {
        PyGC_Head *gc = _PyGCHead_NEXT(reachable);
        assert(gc_old_space(gc) == visited_space);
        gc_list_move(gc, visited);
        PyObject *op = FROM_GC(gc);
        traverseproc traverse = Py_TYPE(op)->tp_traverse;
        (void) traverse(op, visit_add_to_container, &arg);
    }
    gc_list_validate_space(visited, visited_space);
    return arg.size;
}
```



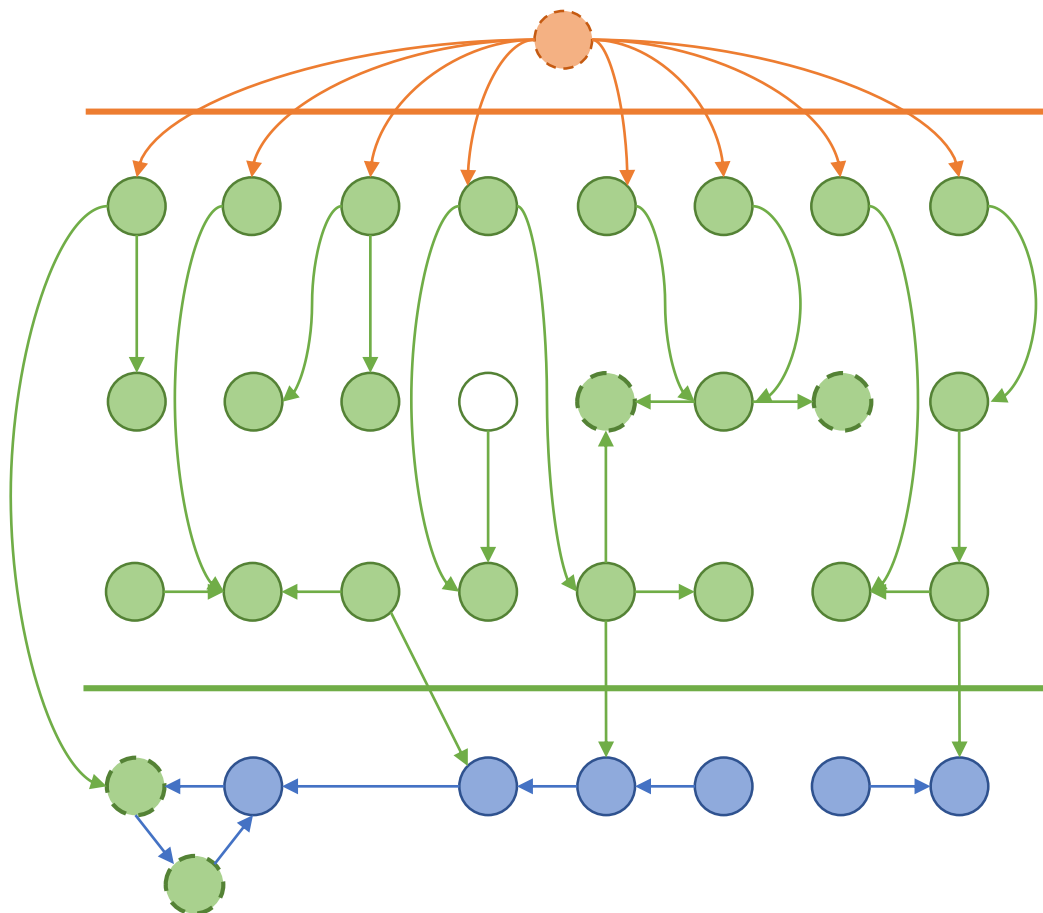
```
static intptr_t
mark_all_reachable(PyGC_Head *reachable, PyGC_Head
*visited, int visited_space)
{
    struct container_and_flag arg = {
        .container = reachable,
        .visited_space = visited_space,
        .size = 0
    };
    while (!gc_list_is_empty(reachable)) {
        PyGC_Head *gc = _PyGCHead_NEXT(reachable);
        assert(gc_old_space(gc) == visited_space);
        gc_list_move(gc, visited);
        PyObject *op = FROM_GC(gc);
        traverseproc traverse = Py_TYPE(op)->tp_traverse;
        (void) traverse(op, visit_add_to_container, &arg);
    }
    gc_list_validate_space(visited, visited_space);
    return arg.size;
}
```



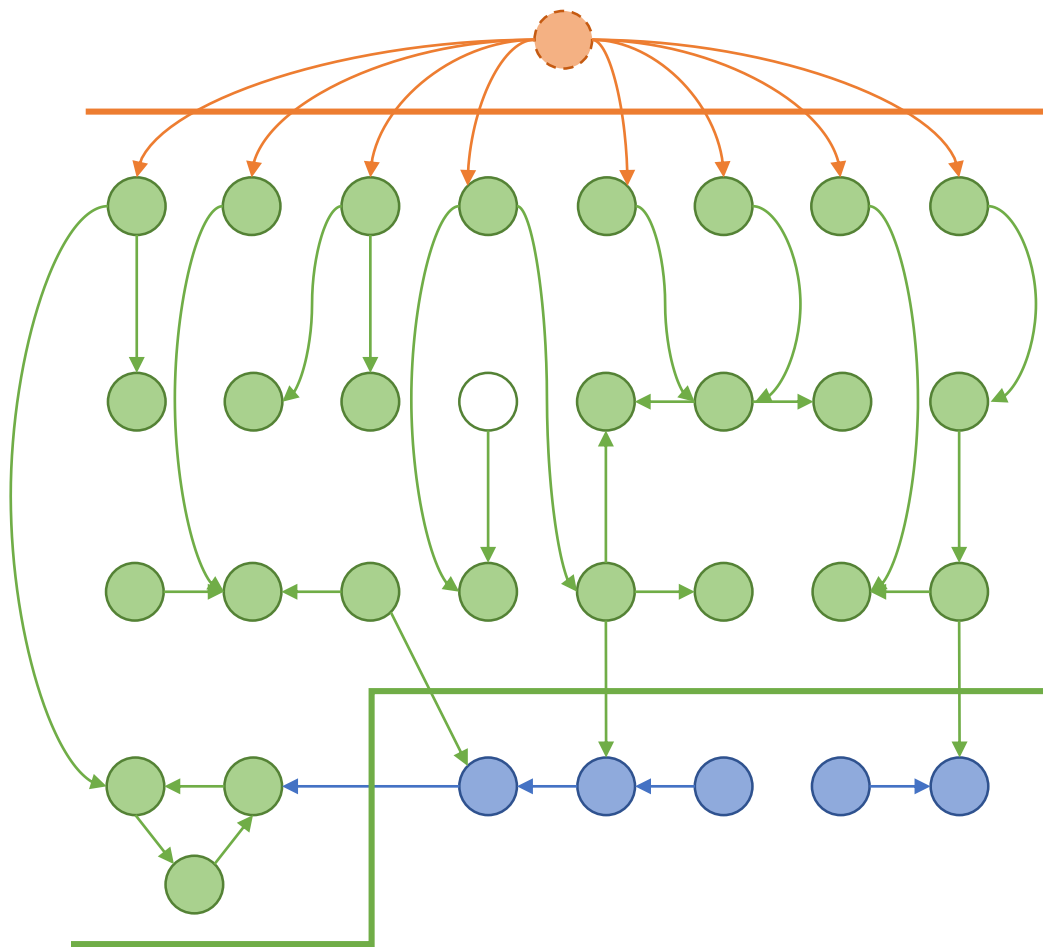
```
static int
visit_add_to_container(PyObject *op, void *arg)
{
    container_and_flag *cf = (container_and_flag *)arg;
    // ...
    if (_PyObject_GC_IS_TRACKED(op) &&
        gc_old_space(gc) != visited) {
        gc_flip_old_space(gc);
        gc_list_move(gc, cf->container);
        cf->size++;
    }
    // ...
}
```



```
static intptr_t
mark_all_reachable(PyGC_Head *reachable, PyGC_Head
*visited, int visited_space)
{
    struct container_and_flag arg = {
        .container = reachable,
        .visited_space = visited_space,
        .size = 0
    };
    while (!gc_list_is_empty(reachable)) {
        PyGC_Head *gc = _PyGCHead_NEXT(reachable);
        assert(gc_old_space(gc) == visited_space);
        gc_list_move(gc, visited);
        PyObject *op = FROM_GC(gc);
        traverseproc traverse = Py_TYPE(op)->tp_traverse;
        (void) traverse(op, visit_add_to_container, &arg);
    }
    gc_list_validate_space(visited, visited_space);
    return arg.size;
}
```



```
static intptr_t
mark_all_reachable(PyGC_Head *reachable, PyGC_Head
*visited, int visited_space)
{
    struct container_and_flag arg = {
        .container = reachable,
        .visited_space = visited_space,
        .size = 0
    };
    while (!gc_list_is_empty(reachable)) {
        PyGC_Head *gc = _PyGCHead_NEXT(reachable);
        assert(gc_old_space(gc) == visited_space);
        gc_list_move(gc, visited);
        PyObject *op = FROM_GC(gc);
        traverseproc traverse = Py_TYPE(op)->tp_traverse;
        (void) traverse(op, visit_add_to_container, &arg);
    }
    gc_list_validate_space(visited, visited_space);
    return arg.size;
}
```



```
static intptr_t
mark_all_reachable(PyGC_Head *reachable, PyGC_Head
*visited, int visited_space)
{
    struct container_and_flag arg = {
        .container = reachable,
        .visited_space = visited_space,
        .size = 0
    };
    while (!gc_list_is_empty(reachable)) {
        PyGC_Head *gc = _PyGCHead_NEXT(reachable);
        assert(gc_old_space(gc) == visited_space);
        gc_list_move(gc, visited);
        PyObject *op = FROM_GC(gc);
        traverseproc traverse = Py_TYPE(op)->tp_traverse;
        (void) traverse(op, visit_add_to_container, &arg);
    }
    gc_list_validate_space(visited, visited_space);
    return arg.size;
}
```



```
static intptr_t
mark_stacks(PyInterpreterState *interp, PyGC_Head *visited, int visited_space, bool start)
{
    PyGC_Head reachable;
    gc_list_init(&reachable);
    Py_ssize_t objects_marked = 0;
    // Move all objects on stacks to reachable
    _PyRuntimeState *runtime = &_PyRuntime;
    HEAD_LOCK(runtime);
    PyThreadState* ts = PyInterpreterState_ThreadHead(interp);
    HEAD_UNLOCK(runtime);
    while (ts) {
        _PyInterpreterFrame *frame = ts->current_frame;
        while (frame) {
            if (frame->owner >= FRAME_OWNED_BY_INTERPRETER) {
                frame = frame->previous;
                continue;
            }
            _PyStackRef *locals = frame->localsplus;
            _PyStackRef *sp = frame->stackpointer;
            objects_marked += move_to_reachable(frame->f_locals, &reachable, visited_space);
            PyObject *func = PyStackRef_AsPyObjectBorrow(frame->f_funcobj);
            objects_marked += move_to_reachable(func, &reachable, visited_space);
            while (sp > locals) {
                sp--;
                if (PyStackRef_IsNullOrInt(*sp)) {
                    continue;
                }
                PyObject *op = PyStackRef_AsPyObjectBorrow(*sp);
                objects_marked += move_to_reachable(op, &reachable, visited_space);
            }
            if (!start && frame->visited) {
                // If this frame has already been visited, then the lower frames
                // will have already been visited and will not have changed
                break;
            }
            frame->visited = 1;
            frame = frame->previous;
        }
        HEAD_LOCK(runtime);
        ts = PyThreadState_Next(ts);
        HEAD_UNLOCK(runtime);
    }
    objects_marked += mark_all_reachable(&reachable, visited, visited_space);
    assert(gc_list_is_empty(&reachable));
    return objects_marked;
}
```

```
static intptr_t
mark_stacks(PyInterpreterState *interp, PyGC_Head *visited, int visited_space,
bool start)
{
    // ...
    _PyRuntimeState *runtime = &_PyRuntime;
    HEAD_LOCK(runtime);
    PyThreadState* ts = PyInterpreterState_ThreadHead(interp);
    HEAD_UNLOCK(runtime);
    while (ts) {
        _PyInterpreterFrame *frame = ts->current_frame;
        while (frame) {
            if (frame->owner >= FRAME_OWNED_BY_INTERPRETER) {
                frame = frame->previous;
                continue;
            }
            // ...
            if (!start && frame->visited) {
                break;
            }
            frame->visited = 1;
            frame = frame->previous;
        }
        HEAD_LOCK(runtime);
        ts = PyThreadState_Next(ts);
        HEAD_UNLOCK(runtime);
    }
    // ...
}
```

```
struct _PyInterpreterFrame {
    _PyStackRef f_executable;
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_globals;
    PyObject *f_builtins;
    PyObject *f_locals;
    PyFrameObject *frame_obj;
    _Py_CODEUNIT *instr_ptr;
    _PyStackRef *stackpointer;
    uint16_t return_offset;
    char owner;
    uint8_t visited;
    /* Locals and stack */
    _PyStackRef localsplus[1];
};
```

```
struct _PyInterpreterFrame {
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_locals;
    _PyStackRef *stackpointer;
    _PyStackRef localsplus[1];
};
```

```
struct _PyInterpreterFrame {
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_locals;
    _PyStackRef *stackpointer;
    _PyStackRef localsplus[1];
};
```

```
static intptr_t
mark_stacks(PyInterpreterState *interp, PyGC_Head *visited, int visited_space, bool start)
{
    // ...
    _PyStackRef *locals = frame->localsplus;
    _PyStackRef *sp = frame->stackpointer;
    objects_marked += move_to_reachable(frame->f_locals, &reachable, visited_space);
    PyObject *func = PyStackRef_AsPyObjectBorrow(frame->f_funcobj);
    objects_marked += move_to_reachable(func, &reachable, visited_space);
    while (sp > locals) {
        sp--;
        if (PyStackRef_IsNullOrInt(*sp)) {
            continue;
        }
        PyObject *op = PyStackRef_AsPyObjectBorrow(*sp);
        objects_marked += move_to_reachable(op, &reachable, visited_space);
    }
    // ...
}
```

```
static intptr_t
mark_stacks(PyInterpreterState *interp, PyGC_Head *visited, int visited_space, bool start)
{
    // ...
    _PyStackRef *locals = frame->localsplus;
    _PyStackRef *sp = frame->stackpointer;
    objects_marked += move_to_reachable(frame->f_locals, &reachable, visited_space);
    PyObject *func = PyStackRef_AsPyObjectBorrow(frame->f_funcobj);
    objects_marked += move_to_reachable(func, &reachable, visited_space);
    while (sp > locals) {
        sp--;
        if (PyStackRef_IsNullOrInt(*sp)) {
            continue;
        }
        PyObject *op = PyStackRef_AsPyObjectBorrow(*sp);
        objects_marked += move_to_reachable(op, &reachable, visited_space);
    }
    // ...
}
```

```
struct _PyInterpreterFrame {
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_locals;
    _PyStackRef *stackpointer;
    _PyStackRef localsplus[1];
};
```

```
sp[ 0]: 0x000001e3023107e0 {bits=1 }
sp[-1]: 0x000001e3023107d8 {bits=2074507934240 }
sp[-2]: 0x000001e3023107d0 {bits=1 }
sp[-3]: 0x000001e3023107c8 {bits=2074510203201 }
sp[-4]: 0x000001e3023107c0 {bits=2074510395137 }
sp[-5]: 0x000001e3023107b8 {bits=2074510441217 }
sp[-6]: 0x000001e3023107b0 {bits=2074506893697 }
locals: 0x000001e3023107b0 {bits=2074506893697 }
```

```
static intptr_t
mark_stacks(PyInterpreterState *interp, PyGC_Head *visited, int visited_space, bool start)
{
    // ...
    objects_marked += mark_all_reachable(&reachable, visited, visited_space);
    assert(gc_list_is_empty(&reachable));
    return objects_marked;
}
```

ФАЗА СБОРКИ

```
static void
gc_collect_increment(PyThreadState *tstate, struct gc_collection_stats *stats)
{
    GCState *gcstate = &tstate->interp->gc;
    gcstate->work_to_do += assess_work_to_do(gcstate);
    untrack_tuples(&gcstate->young.head);
    if (gcstate->phase == GC_PHASE_MARK) {
        //...
        return;
    }
    PyGC_Head *not_visited = &gcstate->old[gcstate->visited_space^1].head;
    PyGC_Head *visited = &gcstate->old[gcstate->visited_space].head;
    PyGC_Head increment;
    gc_list_init(&increment);
    int scale_factor = max(2, gcstate->old[0].threshold);
    intptr_t objects_marked = mark_stacks(tstate->interp, visited, gcstate->visited_space, false);
    gcstate->work_to_do -= objects_marked;
    gc_list_set_space(&gcstate->young.head, gcstate->visited_space);
    gc_list_merge(&gcstate->young.head, &increment);
    Py_ssize_t increment_size = gc_list_size(&increment);
    while (increment_size < gcstate->work_to_do) {
        if (gc_list_is_empty(not_visited)) {
            break;
        }
        PyGC_Head *gc = _PyGCHead_NEXT(not_visited);
        gc_list_move(gc, &increment);
        increment_size++;
        gc_set_old_space(gc, gcstate->visited_space);
        increment_size += expand_region_transitively_reachable(&increment, gc, gcstate);
    }
    PyGC_Head survivors;
    gc_list_init(&survivors);
    gc_collect_region(tstate, &increment, &survivors, stats);
    gc_list_merge(&survivors, visited);
    gcstate->work_to_do += gcstate->heap_size / SCAN_RATE_DIVISOR / scale_factor;
    gcstate->work_to_do -= increment_size;

    add_stats(gcstate, 1, stats);
    if (gc_list_is_empty(not_visited)) {
        completed_scarvenge(gcstate);
    }
}
```

1
2
3

```
static void
gc_collect_increment(PyThreadState *tstate, struct gc_collection_stats *stats)
{
    // ...
    intptr_t objects_marked = mark_stacks(tstate->interp, visited, gcstate->visited_space, false);
    gcstate->work_to_do -= objects_marked;
    // ...
}
```



```
static intptr_t
mark_stacks(PyInterpreterState *interp, PyGC_Head *visited, int visited_space,
bool start)
{
    // ...
    _PyRuntimeState *runtime = &_amp;PyRuntime;
    HEAD_LOCK(runtime);
    PyThreadState* ts = PyInterpreterState_ThreadHead(interp);
    HEAD_UNLOCK(runtime);
    while (ts) {
        _PyInterpreterFrame *frame = ts->current_frame;
        while (frame) {
            if (frame->owner >= FRAME_OWNED_BY_INTERPRETER) {
                frame = frame->previous;
                continue;
            }
            // ...
            if (!start && frame->visited) {
                break;
            }
            frame->visited = 1;
            frame = frame->previous;
        }
        HEAD_LOCK(runtime);
        ts = PyThreadState_Next(ts);
        HEAD_UNLOCK(runtime);
    }
    // ...
}
```

```
struct _PyInterpreterFrame {
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_locals;
    _PyStackRef *stackpointer;
    uint8_t visited; // = 1
    _PyStackRef localsplus[1];
};
```

```
struct _PyInterpreterFrame {
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_locals;
    _PyStackRef *stackpointer;
    uint8_t visited; // = 1
    _PyStackRef localsplus[1];
};
```

```
struct _PyInterpreterFrame {
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_locals;
    _PyStackRef *stackpointer;
    uint8_t visited; // = 0
    _PyStackRef localsplus[1];
};
```

```
struct _PyInterpreterFrame {
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_locals;
    _PyStackRef *stackpointer;
    uint8_t visited; // = 0
    _PyStackRef localsplus[1];
};
```

```
struct _PyInterpreterFrame {
    struct _PyInterpreterFrame *previous;
    _PyStackRef f_funcobj;
    PyObject *f_locals;
    _PyStackRef *stackpointer;
    uint8_t visited; // = 0
    _PyStackRef localsplus[1];
};
```



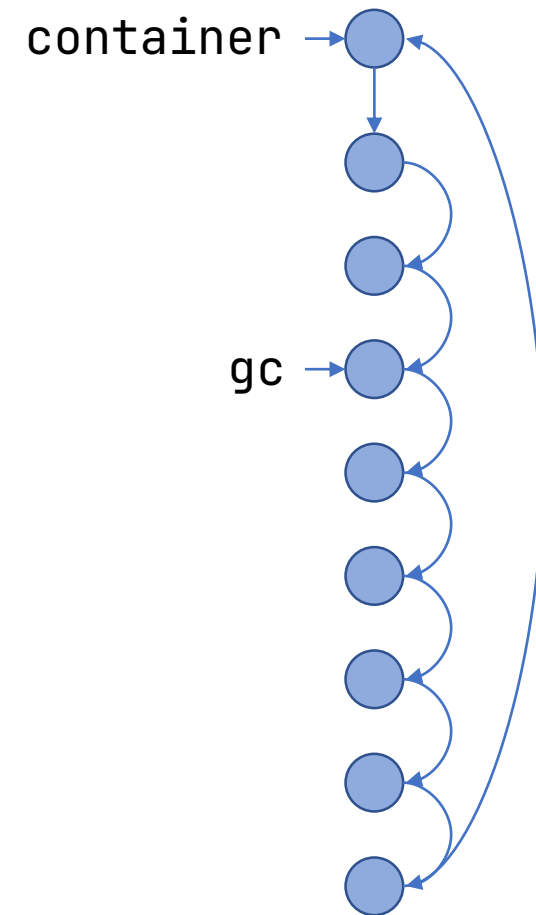
```
static void
gc_collect_increment(PyThreadState *tstate, struct gc_collection_stats *stats)
{
    GCState *gcstate = &tstate->interp->gc;
    gcstate->work_to_do += assess_work_to_do(gcstate);
    // ...
    PyGC_Head increment;
    gc_list_init(&increment);
    // ...
    gc_list_merge(&gcstate->young.head, &increment);
    Py_ssize_t increment_size = gc_list_size(&increment);
    while (increment_size < gcstate->work_to_do) {
        if (gc_list_is_empty(not_visited)) {
            break;
        }
        PyGC_Head *gc = _PyGCHead_NEXT(not_visited);
        gc_list_move(gc, &increment);
        increment_size++;
        gc_set_old_space(gc, gcstate->visited_space);
        increment_size += expand_region_transitively_reachable(&increment, gc, gcstate);
    }
    // ...
}
```

```

static intptr_t
expand_region_transitively_reachable(PyGC_Head *container, PyGC_Head *gc, GCState *gcstate)
{
    struct container_and_flag arg = {
        .container = container,
        .visited_space = gcstate->visited_space,
        .size = 0
    };
    while (gc != container) {
        // ...
        traverseproc traverse = Py_TYPE(op)->tp_traverse;
        (void) traverse(op, visit_add_to_container, &arg);
        gc = GC_NEXT(gc);
    }
    // ...
}

static int
visit_add_to_container(PyObject *op, void *arg)
{
    // ...
    if (_PyObject_GC_IS_TRACKED(op) && gc_old_space(gc) != visited) {
        gc_flip_old_space(gc);
        gc_list_move(gc, cf->container);
        cf->size++;
    }
    // ...
}

```

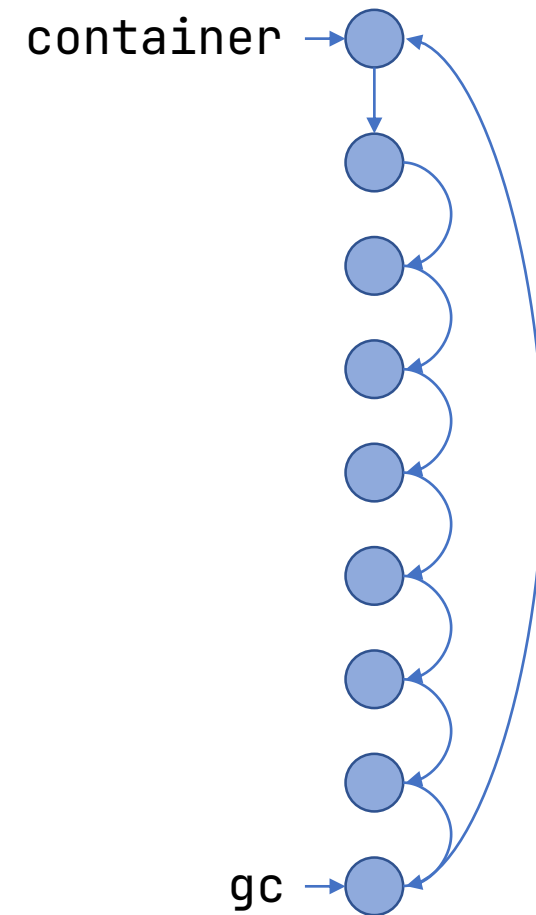


```

static intptr_t
expand_region_transitively_reachable(PyGC_Head *container, PyGC_Head *gc, GCState *gcstate)
{
    struct container_and_flag arg = {
        .container = container,
        .visited_space = gcstate->visited_space,
        .size = 0
    };
    while (gc != container) {
        // ...
        traverseproc traverse = Py_TYPE(op)->tp_traverse;
        (void) traverse(op, visit_add_to_container, &arg);
        gc = GC_NEXT(gc);
    }
    // ...
}

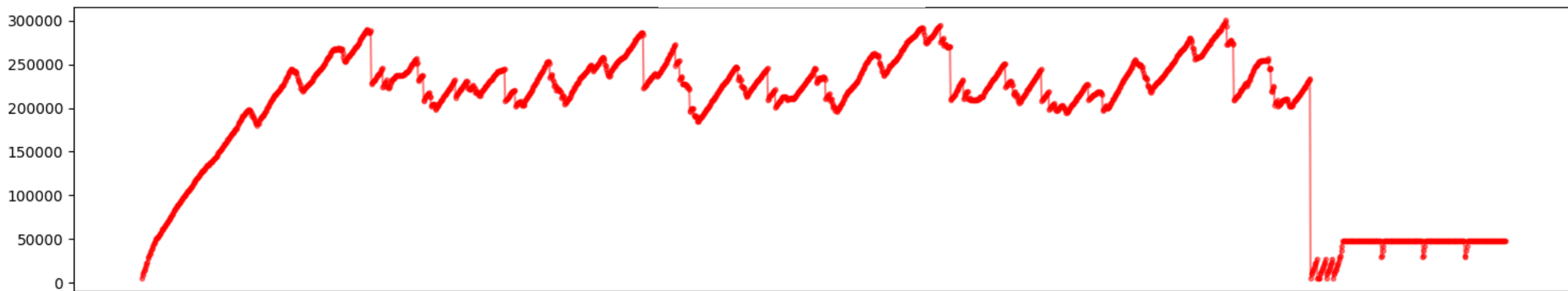
static int
visit_add_to_container(PyObject *op, void *arg)
{
    // ...
    if (_PyObject_GC_IS_TRACKED(op) && gc_old_space(gc) != visited) {
        gc_flip_old_space(gc);
        gc_list_move(gc, cf->container);
        cf->size++;
    }
    // ...
}

```

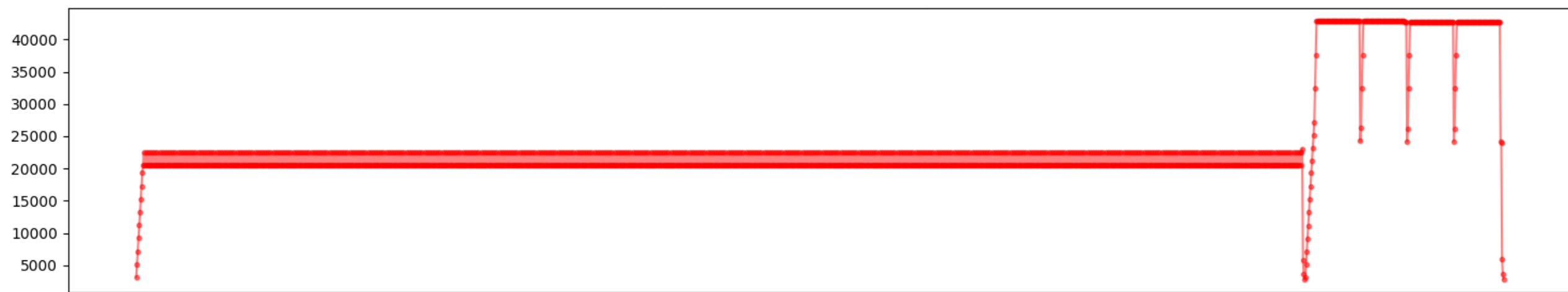


PYPERFORMANCE

heap_size (3.14+)



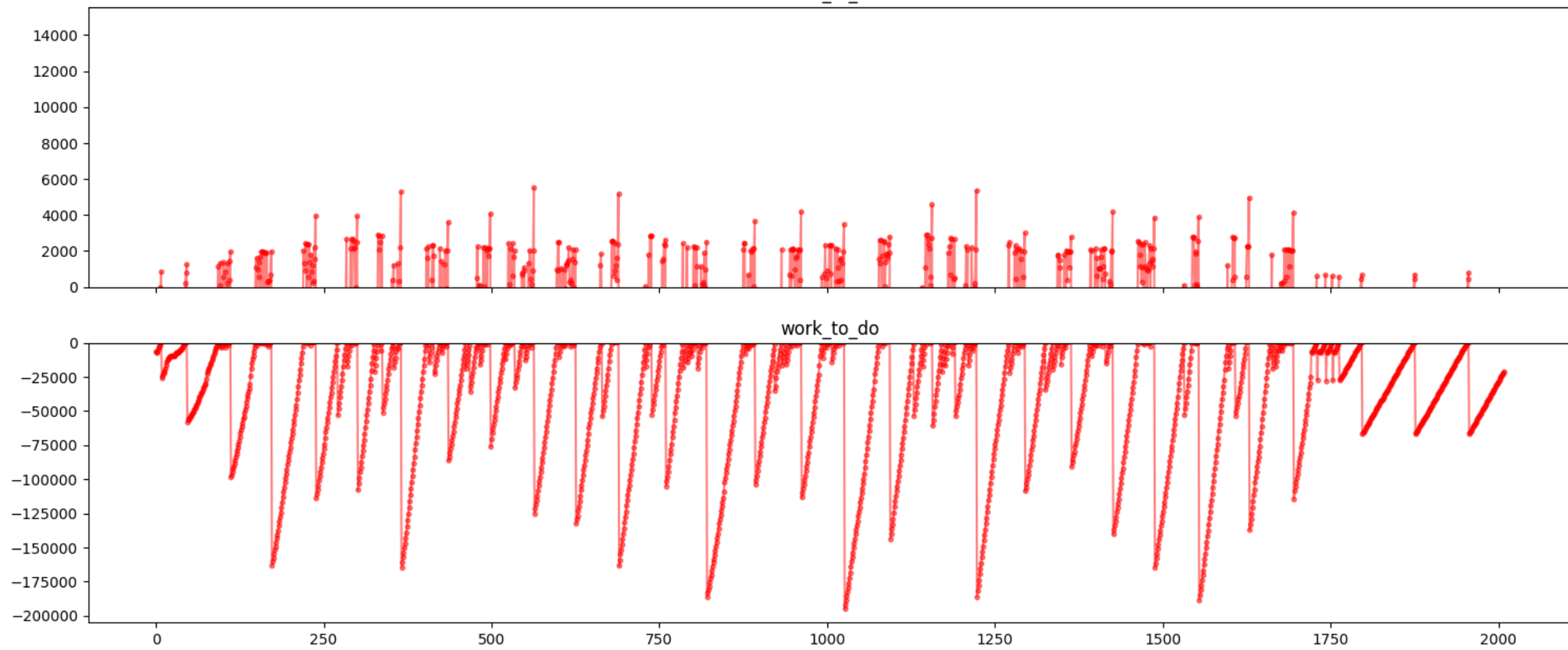
heap_size (3.13)



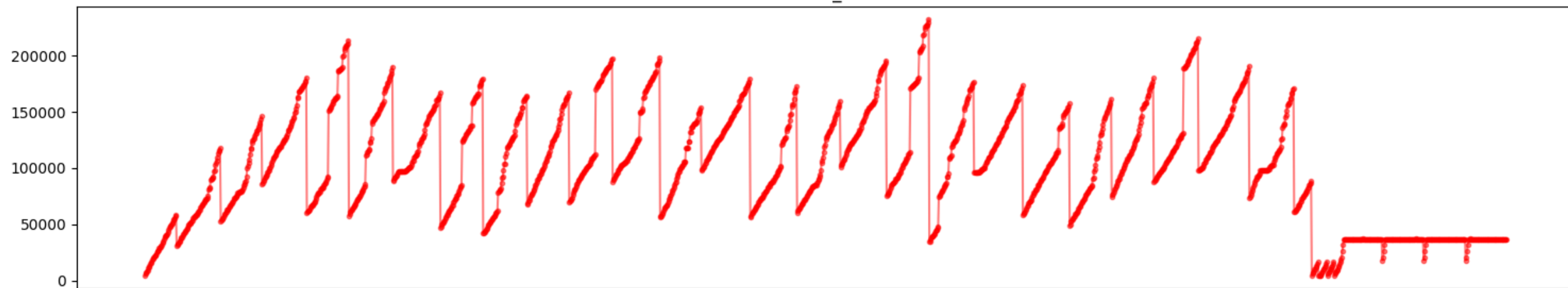
```
static void
gc_collect_increment(PyThreadState *tstate, struct gc_collection_stats *stats)
{
    GC_STAT_ADD(1, collections, 1);
    GCState *gcstate = &tstate->interp->gc;
    gcstate->work_to_do += assess_work_to_do(gcstate);
    // ...
    if (gcstate->phase == GC_PHASE_MARK) {
        Py_ssize_t objects_marked = mark_at_start(tstate);
        gcstate->work_to_do -= objects_marked;
        return;
    }
    intptr_t objects_marked = mark_stacks(tstate->interp, visited, gcstate->visited_space, false);
    gcstate->work_to_do -= objects_marked;
    // ...
    gc_collect_region(tstate, &increment, &survivors, stats);
    gc_list_merge(&survivors, visited);
    gcstate->work_to_do += gcstate->heap_size / SCAN_RATE_DIVISOR / scale_factor;
    gcstate->work_to_do -= increment_size;

    // ...
}
```

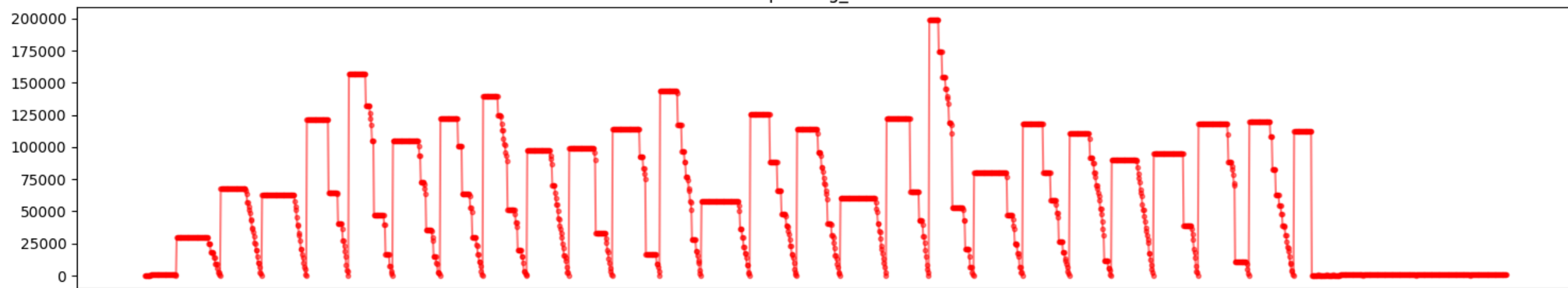
work_to_do



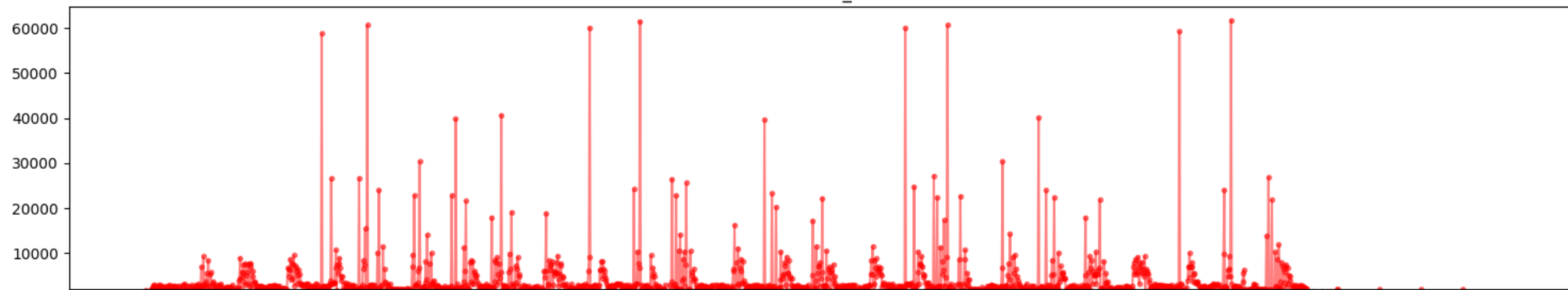
visited_size



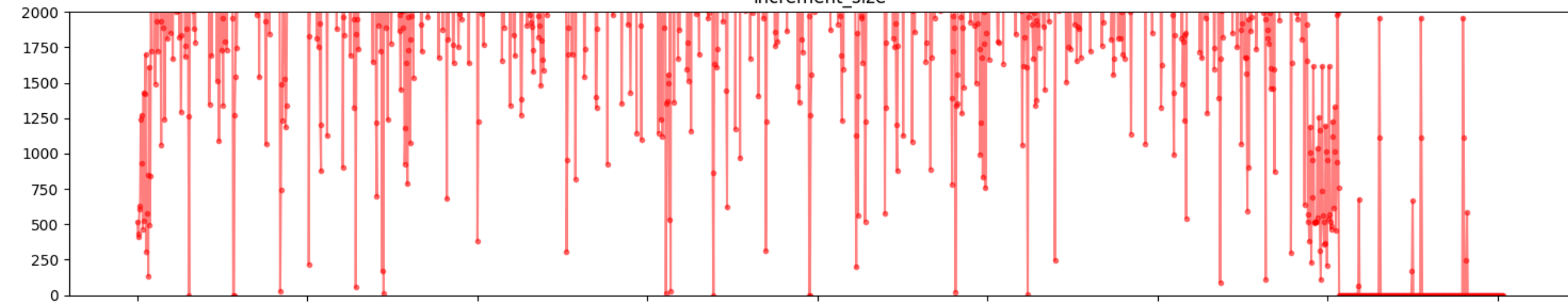
pending_size



increment_size



increment_size



With marking added to the cyclic GC ([#127110](#)) we spend a lot of the time in the GC forming transitive closures, both for marking and for the increments of the incremental GC.

Unfortunately the current algorithm has a couple of mistakes in it.

One harmful, one beneficial.

- The beneficial one is counting the initial mark twice. This helps because it reduces the cost of GC on heaps with little or no garbage
- The harmful one is allowing the amount of work done to grow in proportion to the heap size.

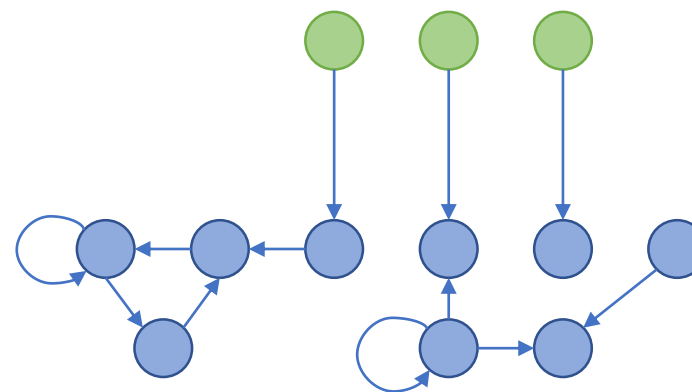
[GH-126491: Lower heap size limit with faster marking by markshannon · Pull Request #127519 · python/cpython](#)

СБОРЩИК МУСОРА

- Поиск недостижимых
- Определение несобираемых
- Обработка слабых ссылок
- Удаление объектов
- Определение воскрешенных объектов
- Очистка слабых ссылок
- Очистка циклов
- Перенос несобираемых

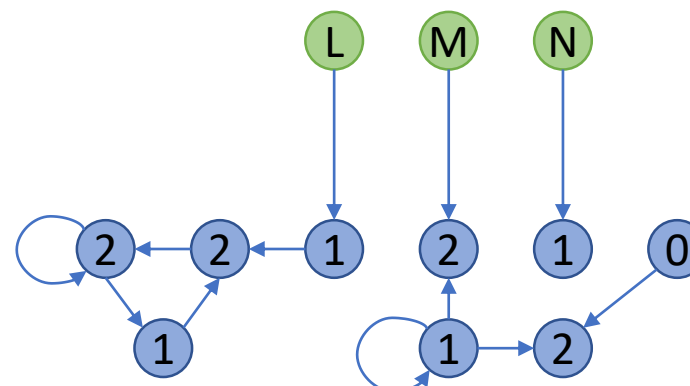
```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```



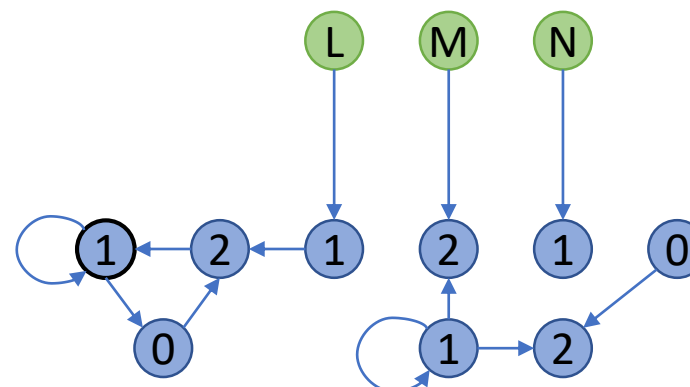
```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```



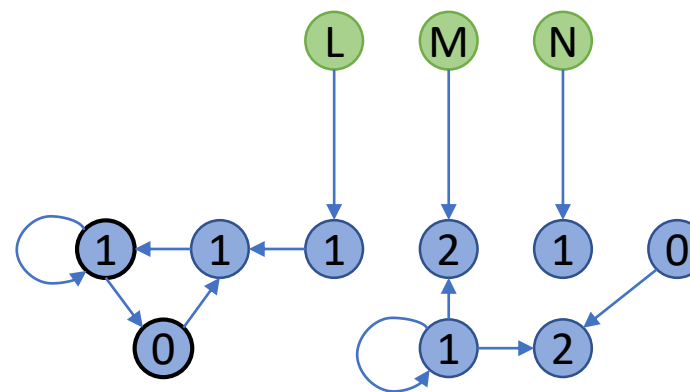
```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```



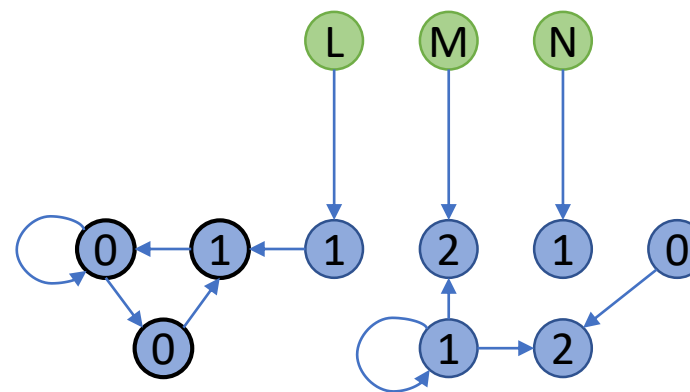

```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```



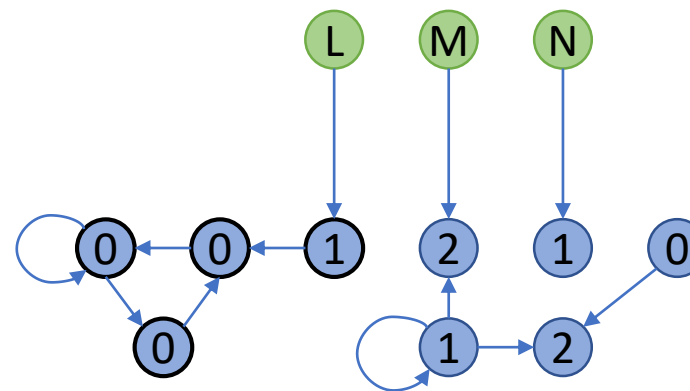
```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```



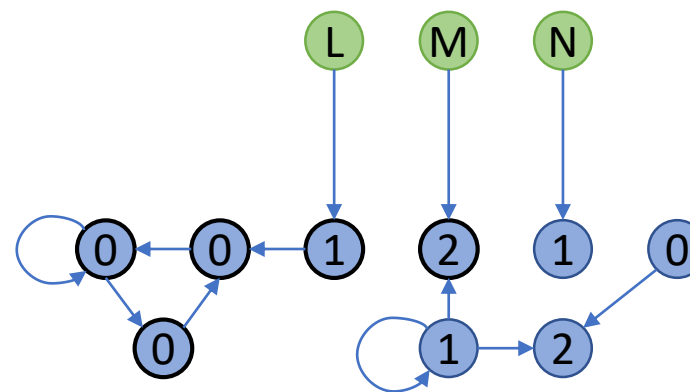
```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```



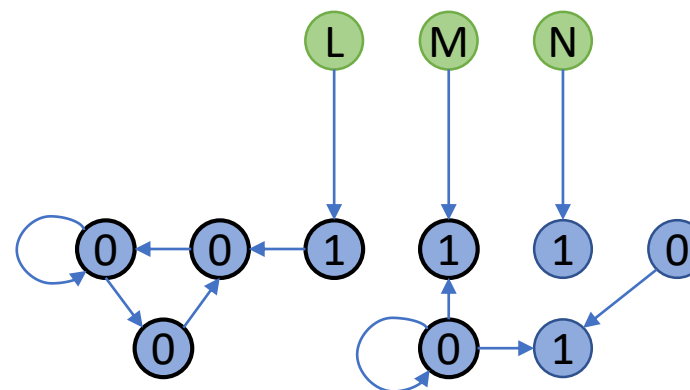
```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```



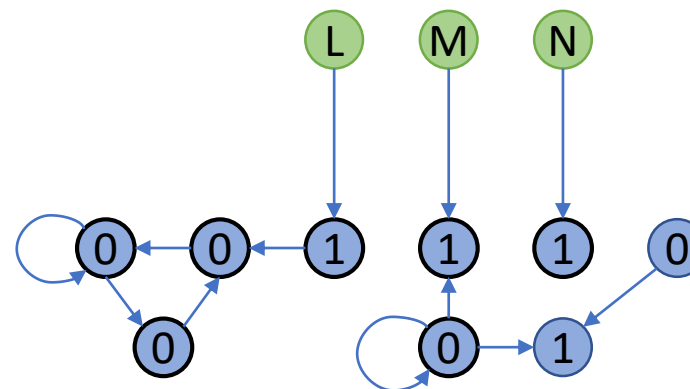
```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```



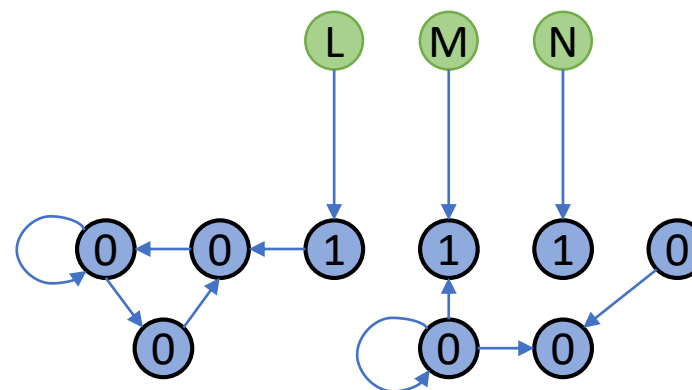
```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```

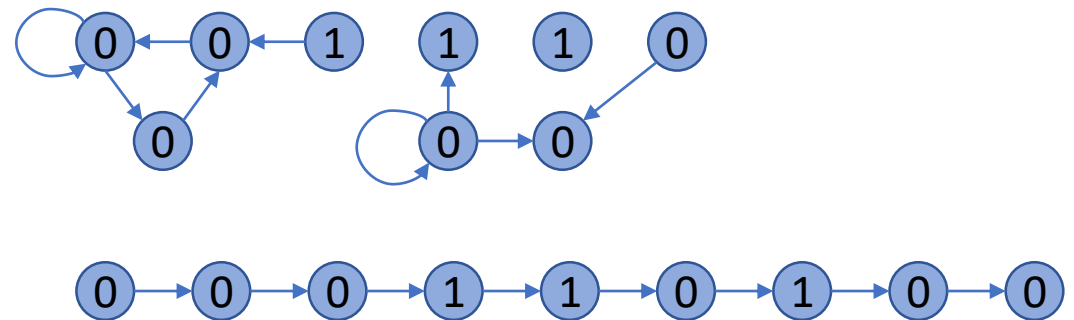



```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

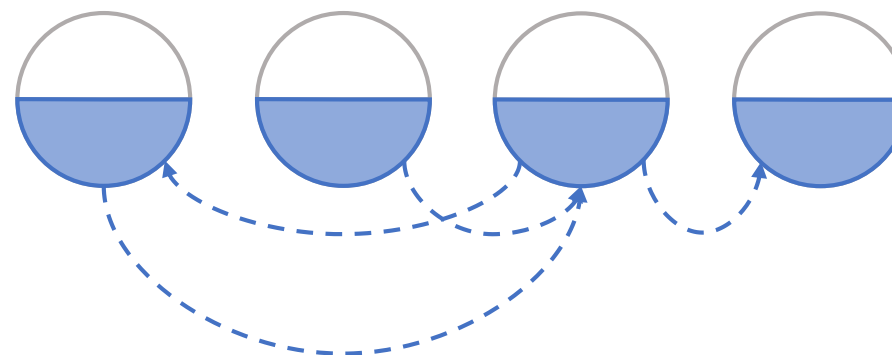
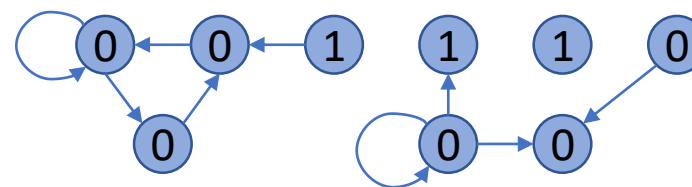
    move_unreachable(base, unreachable);
}
```



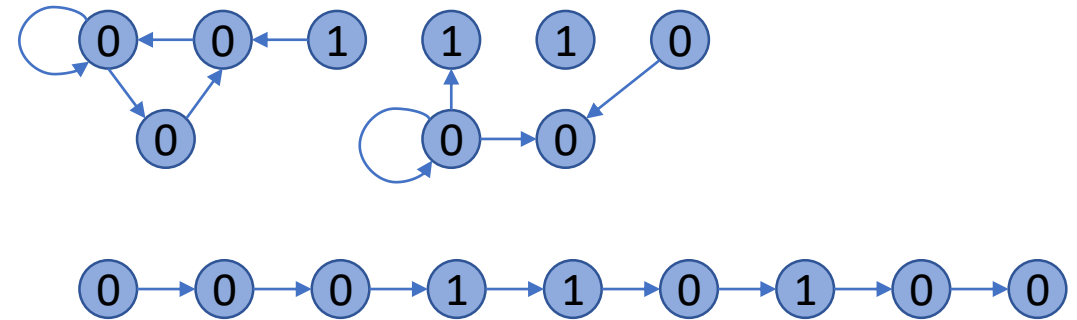

```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            PyObject *op = FROM_GC(gc);
            (void) Py_TYPE(op)->tp_traverse(op,
visit_reachable, (void *)young);
            _PyGCHead_SET_PREV(gc, prev);
            gc_clear_collecting(gc);
            prev = gc;
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



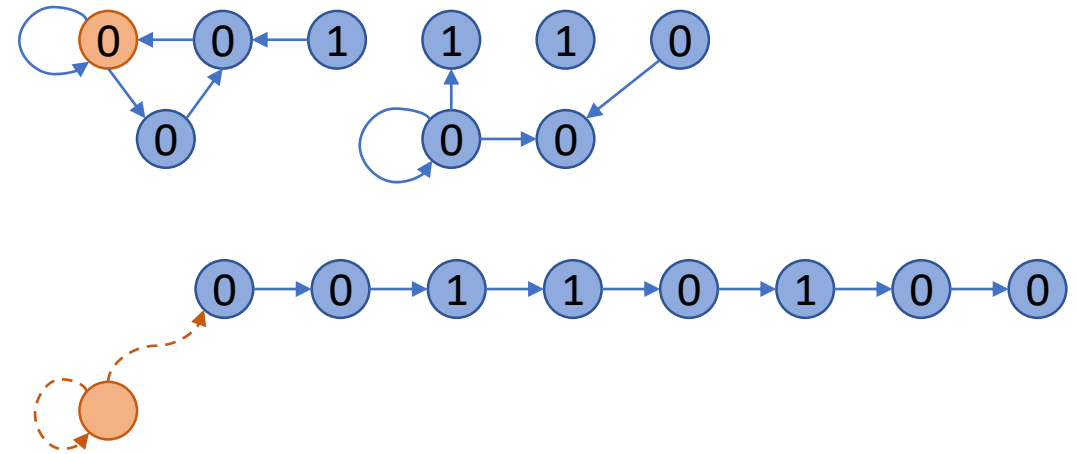
```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            PyObject *op = FROM_GC(gc);
            (void) Py_TYPE(op)->tp_traverse(op,
visit_reachable, (void *)young);
            _PyGCHead_SET_PREV(gc, prev);
            gc_clear_collecting(gc);
            prev = gc;
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



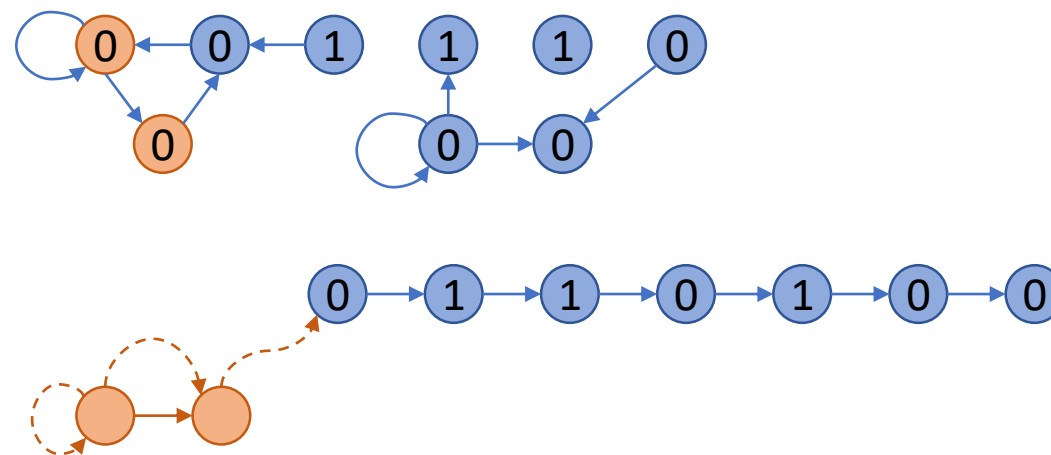

```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            PyObject *op = FROM_GC(gc);
            (void) Py_TYPE(op)->tp_traverse(op,
visit_reachable, (void *)young);
            _PyGCHead_SET_PREV(gc, prev);
            gc_clear_collecting(gc);
            prev = gc;
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



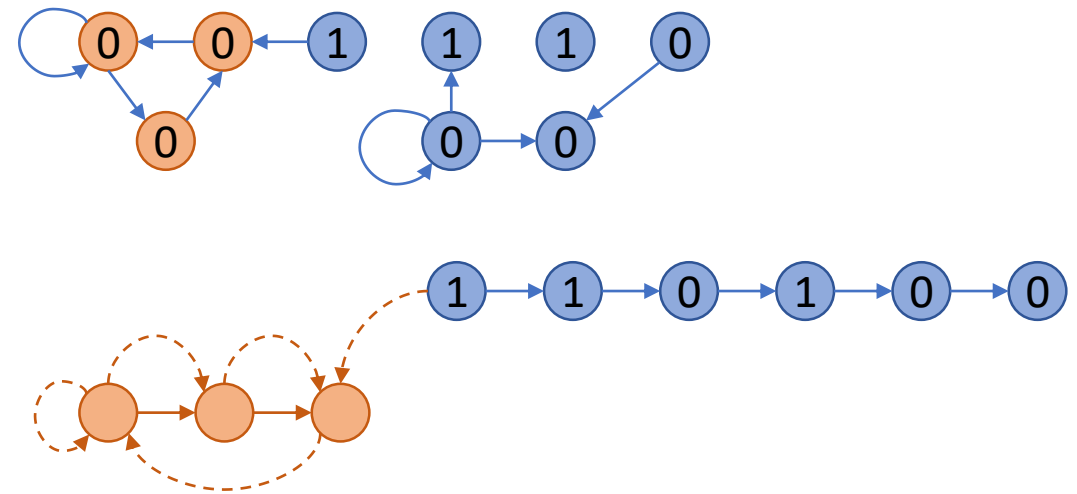
```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            // ...
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



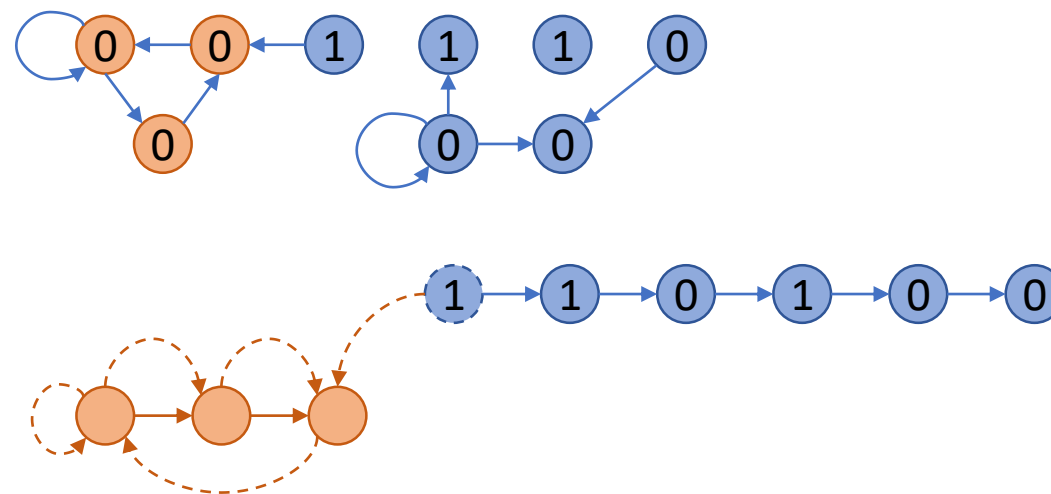
```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            // ...
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            // ...
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



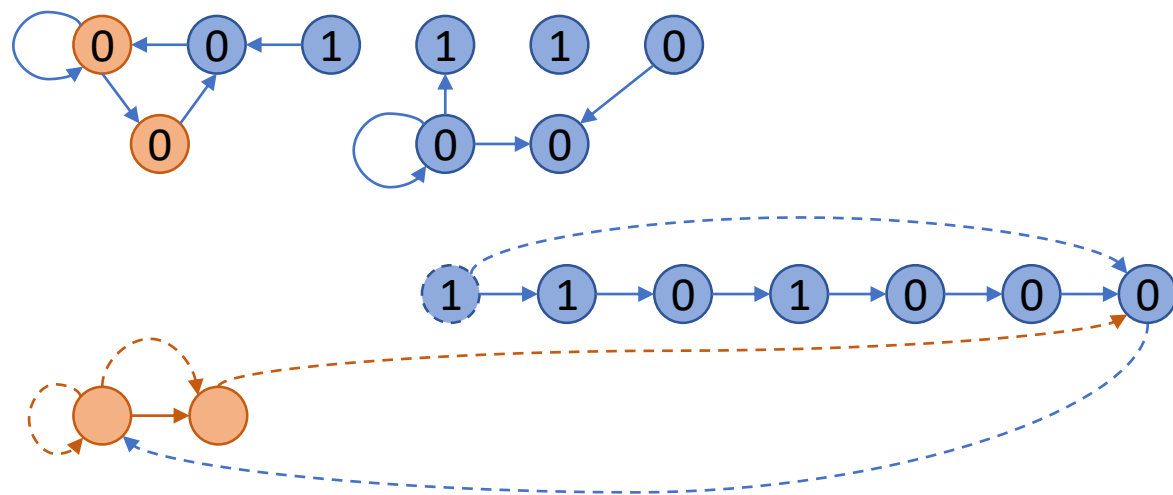

```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    while (gc != young) {
        if (gc_get_refs(gc)) {
            PyObject *op = FROM_GC(gc);
            (void) Py_TYPE(op)->tp_traverse(op,
visit_reachable, (void *)young);
            _PyGCHead_SET_PREV(gc, prev);
            gc_clear_collecting(gc);
            prev = gc;
        }
        else {
            // ...
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



```
static int
visit_reachable(PyObject *op, void *arg)
{
    // ...

    if (gc->_gc_next & NEXT_MASK_UNREACHABLE) {
        PyGC_Head *prev = GC_PREV(gc);
        PyGC_Head *next = GC_NEXT(gc);
        prev->_gc_next = gc->_gc_next;
        gc->_gc_next &= ~NEXT_MASK_UNREACHABLE;
        _PyGCHead_SET_PREV(next, prev);

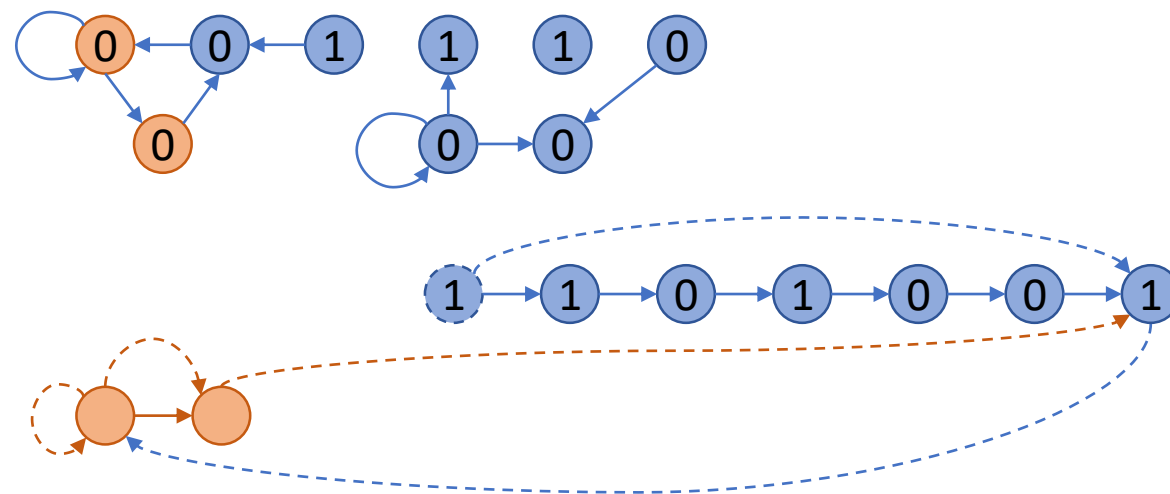
        gc_list_append(gc, reachable);
        gc_set_refs(gc, 1);
    }
    // ...
}
```



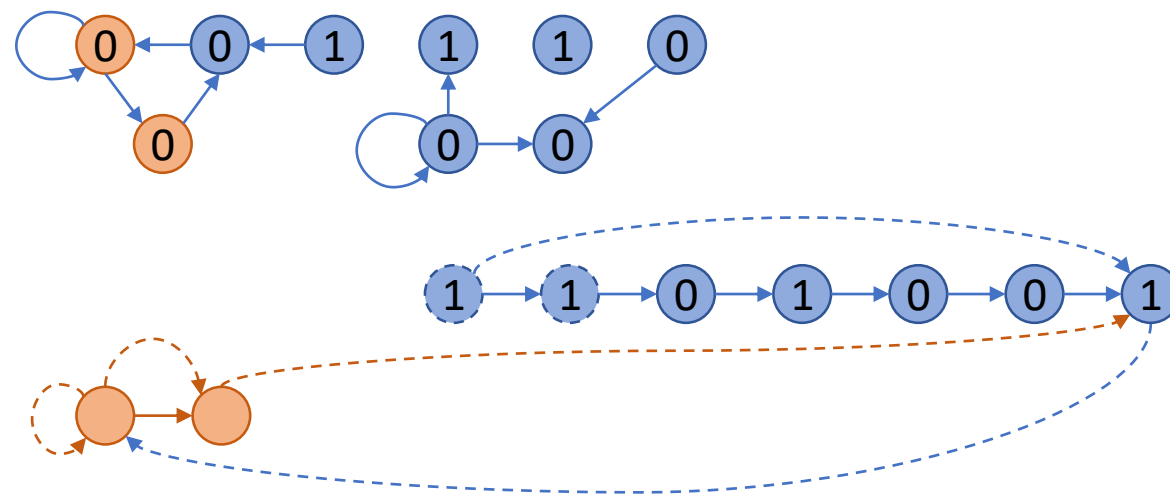
```
static int
visit_reachable(PyObject *op, void *arg)
{
    // ...

    if (gc->_gc_next & NEXT_MASK_UNREACHABLE) {
        PyGC_Head *prev = GC_PREV(gc);
        PyGC_Head *next = GC_NEXT(gc);
        prev->_gc_next = gc->_gc_next;
        gc->_gc_next &= ~NEXT_MASK_UNREACHABLE;
        _PyGCHead_SET_PREV(next, prev);

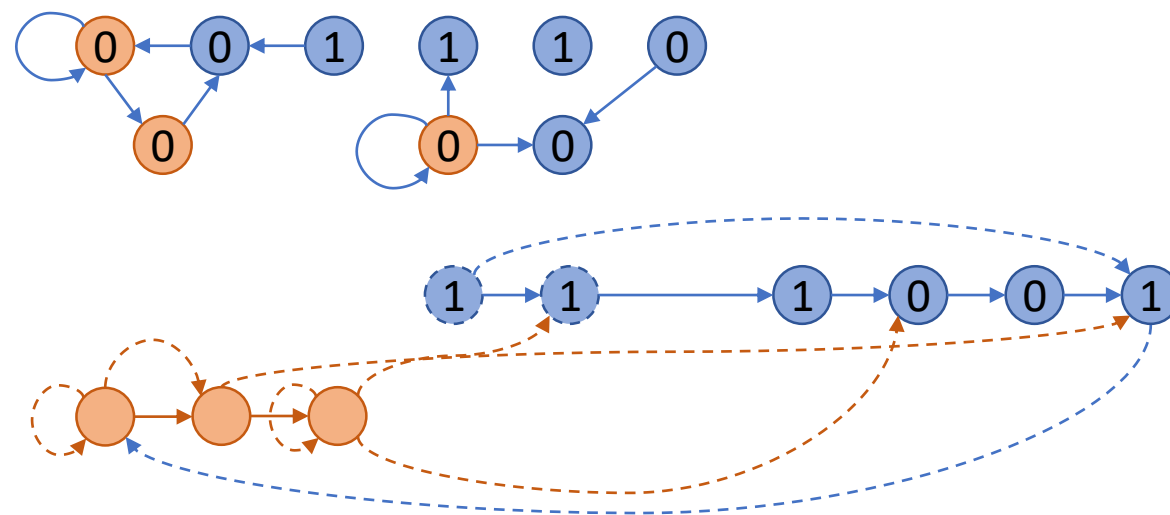
        gc_list_append(gc, reachable);
        gc_set_refs(gc, 1);
    }
    // ...
}
```



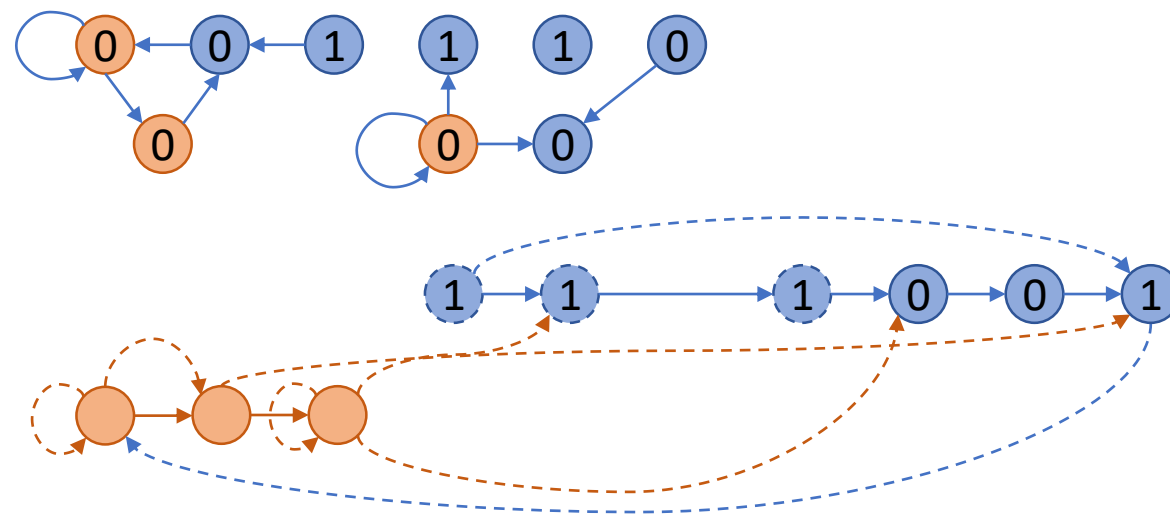
```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    while (gc != young) {
        if (gc_get_refs(gc)) {
            PyObject *op = FROM_GC(gc);
            (void) Py_TYPE(op)->tp_traverse(op,
visit_reachable, (void *)young);
            _PyGCHead_SET_PREV(gc, prev);
            gc_clear_collecting(gc);
            prev = gc;
        }
        else {
            // ...
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



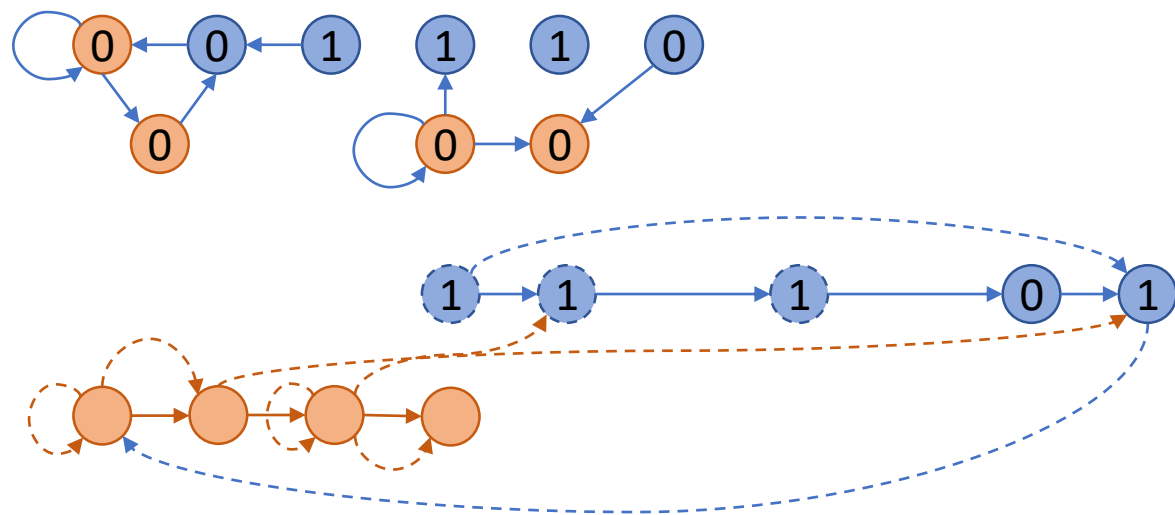
```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            // ...
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



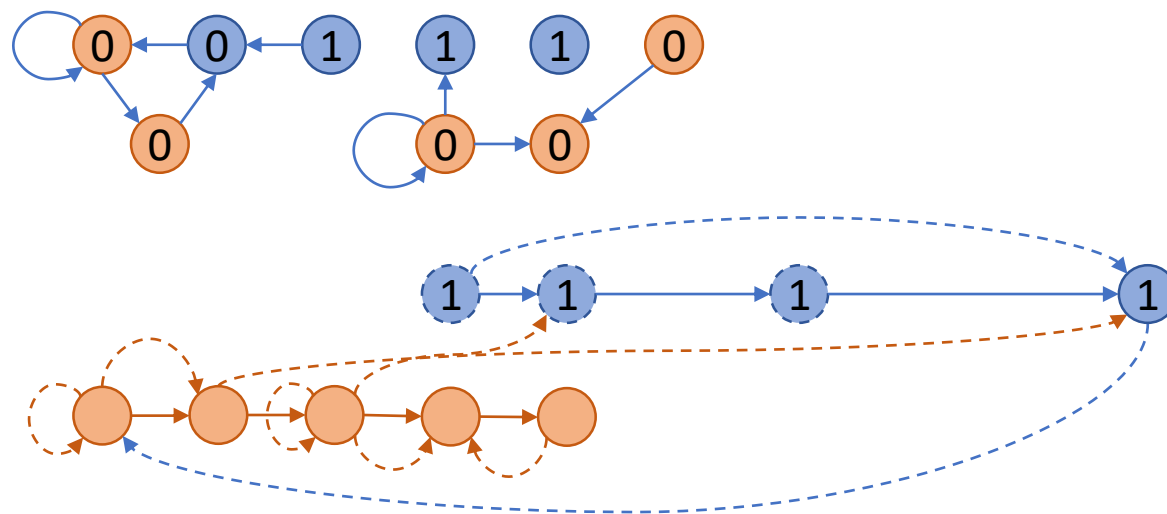
```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    while (gc != young) {
        if (gc_get_refs(gc)) {
            PyObject *op = FROM_GC(gc);
            (void) Py_TYPE(op)->tp_traverse(op,
visit_reachable, (void *)young);
            _PyGCHead_SET_PREV(gc, prev);
            gc_clear_collecting(gc);
            prev = gc;
        }
        else {
            // ...
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



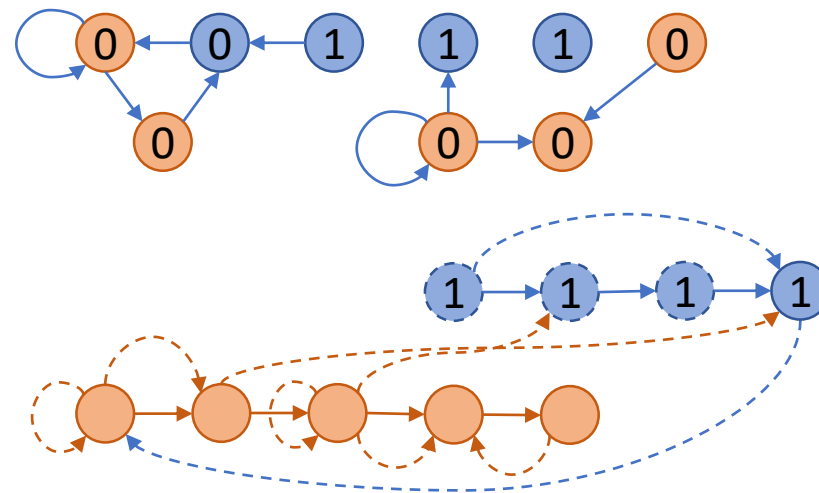
```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            // ...
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            // ...
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



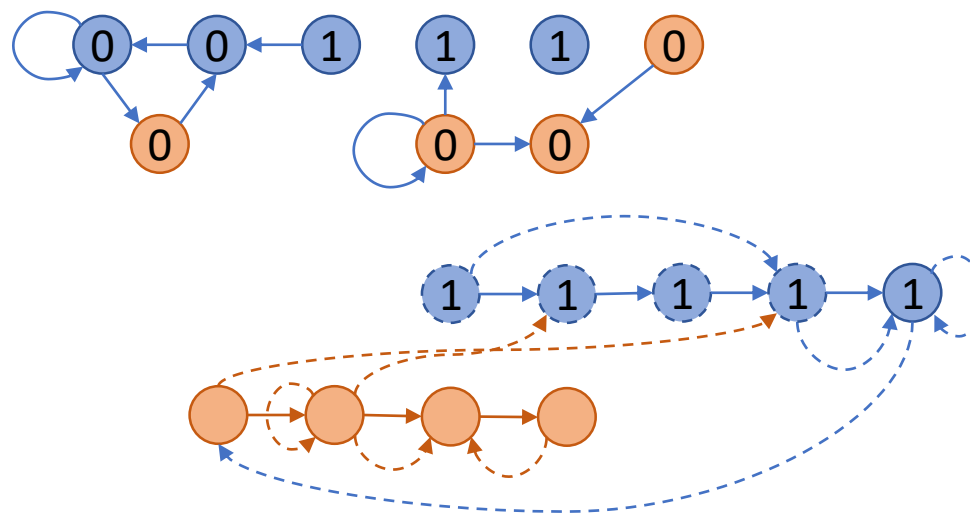

```
static void
move_unreachable(PyGC_Head *young, PyGC_Head
*unreachable)
{
    // ...
    uintptr_t flags = NEXT_MASK_UNREACHABLE | (gc-
>_gc_next & _PyGC_NEXT_MASK_OLD_SPACE_1);
    while (gc != young) {
        if (gc_get_refs(gc)) {
            // ...
        }
        else {
            prev->_gc_next = gc->_gc_next;
            PyGC_Head *last = GC_PREV(unreachable);
            last->_gc_next = flags | gc;
            _PyGCHead_SET_PREV(gc, last);
            gc->_gc_next = flags | unreachable;
            unreachable->_gc_prev = gc;
        }
        gc = _PyGCHead_NEXT(prev);
    }
    // ...
}
```



```
static int
visit_reachable(PyObject *op, void *arg)
{
    // ...

    if (gc->_gc_next & NEXT_MASK_UNREACHABLE) {
        PyGC_Head *prev = GC_PREV(gc);
        PyGC_Head *next = GC_NEXT(gc);
        prev->_gc_next = gc->_gc_next;
        gc->_gc_next &= ~NEXT_MASK_UNREACHABLE;
        _PyGCHead_SET_PREV(next, prev);

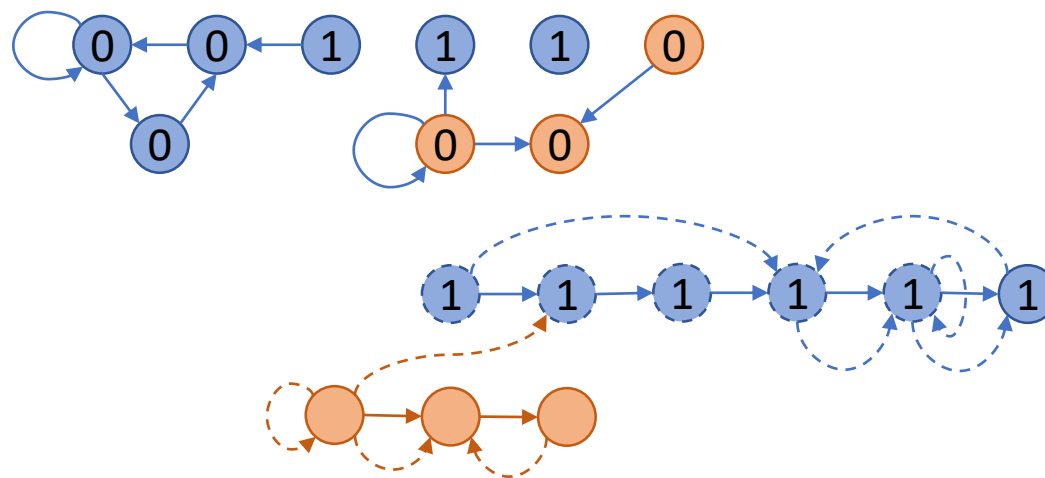
        gc_list_append(gc, reachable);
        gc_set_refs(gc, 1);
    }
    // ...
}
```



```
static int
visit_reachable(PyObject *op, void *arg)
{
    // ...

    if (gc->_gc_next & NEXT_MASK_UNREACHABLE) {
        PyGC_Head *prev = GC_PREV(gc);
        PyGC_Head *next = GC_NEXT(gc);
        prev->_gc_next = gc->_gc_next;
        gc->_gc_next &= ~NEXT_MASK_UNREACHABLE;
        _PyGCHead_SET_PREV(next, prev);

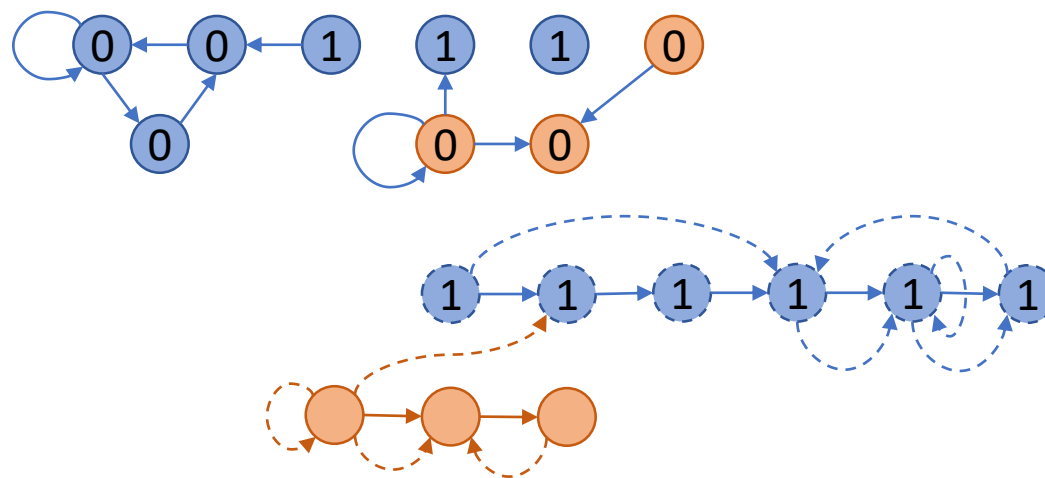
        gc_list_append(gc, reachable);
        gc_set_refs(gc, 1);
    }
    // ...
}
```



```
static int
visit_reachable(PyObject *op, void *arg)
{
    // ...

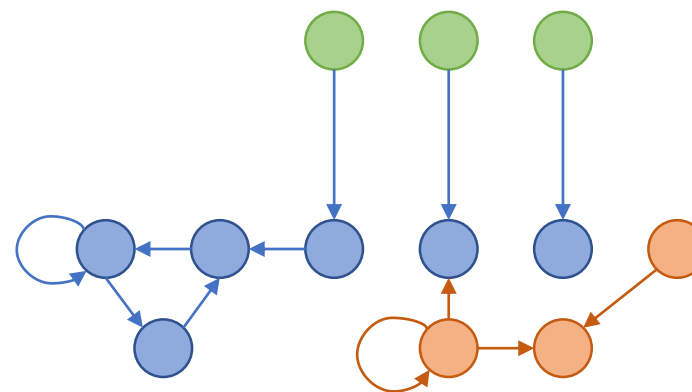
    if (gc->_gc_next & NEXT_MASK_UNREACHABLE) {
        PyGC_Head *prev = GC_PREV(gc);
        PyGC_Head *next = GC_NEXT(gc);
        prev->_gc_next = gc->_gc_next;
        gc->_gc_next &= ~NEXT_MASK_UNREACHABLE;
        _PyGCHead_SET_PREV(next, prev);

        gc_list_append(gc, reachable);
        gc_set_refs(gc, 1);
    }
    else if (gc_refs == 0) {
        gc_set_refs(gc, 1);
    }
    return 0;
}
```



```
static inline void
deduce_unreachable(PyGC_Head *base, PyGC_Head
*unreachable)
{
    update_refs(base);
    subtract_refs(base);

    move_unreachable(base, unreachable);
}
```

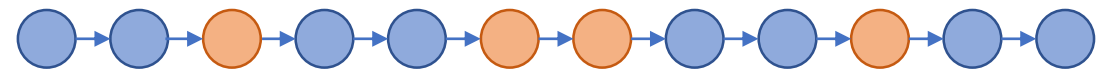


- Поиск недостижимых
- Определение несобираемых
- Обработка слабых ссылок
- Удаление объектов
- Определение воскрешенных объектов
- Очистка слабых ссылок
- Очистка циклов
- Перенос несобираемых

```
static void
move_legacy_finalizers(PyGC_Head *unreachable, PyGC_Head
*finalizers)
{
    PyGC_Head *gc, *next;
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        PyObject *op = FROM_GC(gc);

        next = GC_NEXT(gc);
        gc->_gc_next &= ~NEXT_MASK_UNREACHABLE;

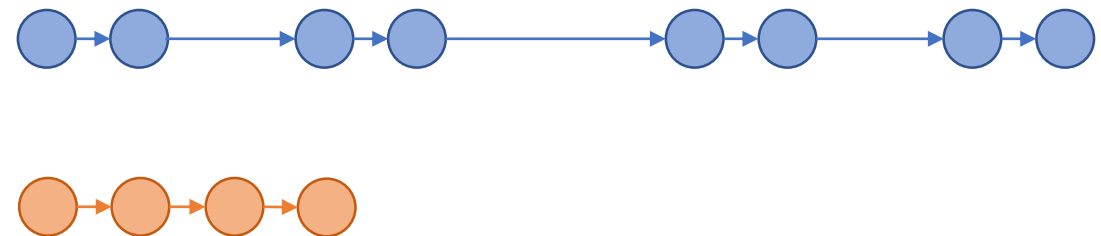
        if (has_legacy_finalizer(op)) {
            gc_clear_collecting(gc);
            gc_list_move(gc, finalizers);
        }
    }
}
```



```
static void
move_legacy_finalizers(PyGC_Head *unreachable, PyGC_Head
*finalizers)
{
    PyGC_Head *gc, *next;
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        PyObject *op = FROM_GC(gc);

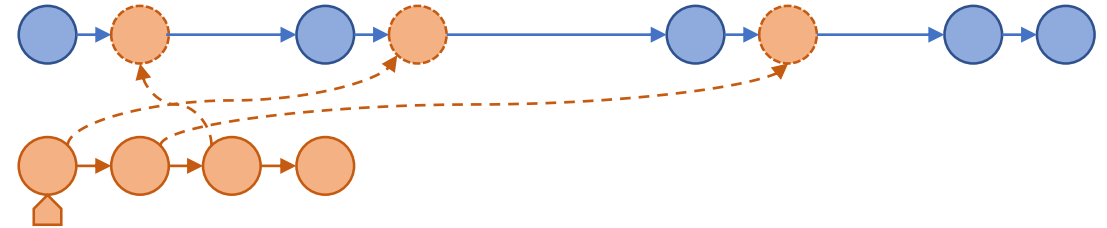
        next = GC_NEXT(gc);
        gc->_gc_next &= ~NEXT_MASK_UNREACHABLE;

        if (has_legacy_finalizer(op)) {
            gc_clear_collecting(gc);
            gc_list_move(gc, finalizers);
        }
    }
}
```



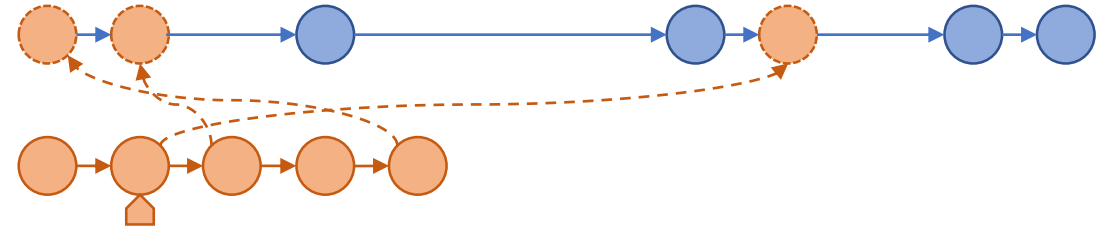

```
static int
visit_move(PyObject *op, void *arg)
{
    // ...
    if (gc_is_collecting(gc)) {
        gc_list_move(gc, tolist);
        gc_clear_collecting(gc);
    }
    // ...
}

static void
move_legacy_finalizer_reachable(PyGC_Head *finalizers)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        (void) Py_TYPE(FROM_GC(gc))->tp_traverse(FROM_GC(gc), visit_move, (void *)finalizers);
    }
}
```



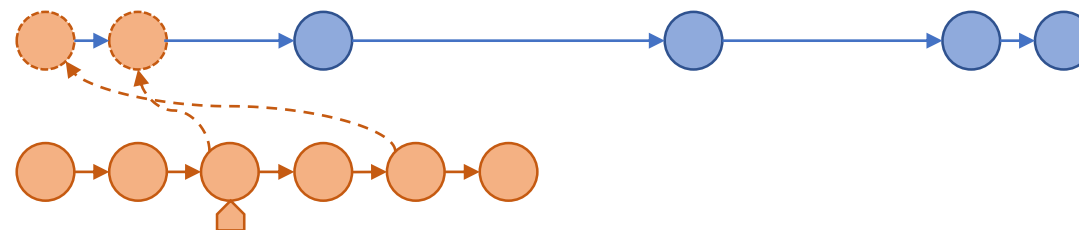
```
static int
visit_move(PyObject *op, void *arg)
{
    // ...
    if (gc_is_collecting(gc)) {
        gc_list_move(gc, tolist);
        gc_clear_collecting(gc);
    }
    // ...
}

static void
move_legacy_finalizer_reachable(PyGC_Head *finalizers)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        (void) Py_TYPE(FROM_GC(gc))->tp_traverse(FROM_GC(gc), visit_move, (void *)finalizers);
    }
}
```



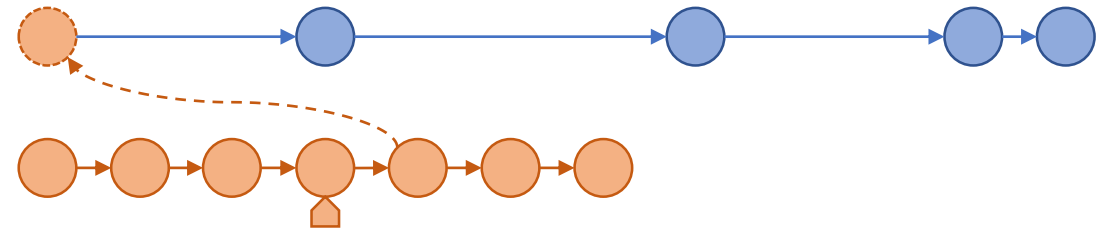
```
static int
visit_move(PyObject *op, void *arg)
{
    // ...
    if (gc_is_collecting(gc)) {
        gc_list_move(gc, tolist);
        gc_clear_collecting(gc);
    }
    // ...
}
```

```
static void
move_legacy_finalizer_reachable(PyGC_Head *finalizers)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        (void) Py_TYPE(FROM_GC(gc))->tp_traverse(FROM_GC(gc), visit_move, (void *)finalizers);
    }
}
```



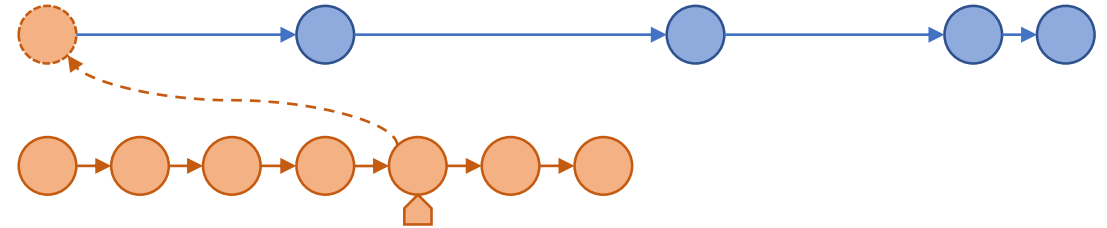
```
static int
visit_move(PyObject *op, void *arg)
{
    // ...
    if (gc_is_collecting(gc)) {
        gc_list_move(gc, tolist);
        gc_clear_collecting(gc);
    }
    // ...
}

static void
move_legacy_finalizer_reachable(PyGC_Head *finalizers)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        (void) Py_TYPE(FROM_GC(gc))->tp_traverse(FROM_GC(gc), visit_move, (void *)finalizers);
    }
}
```



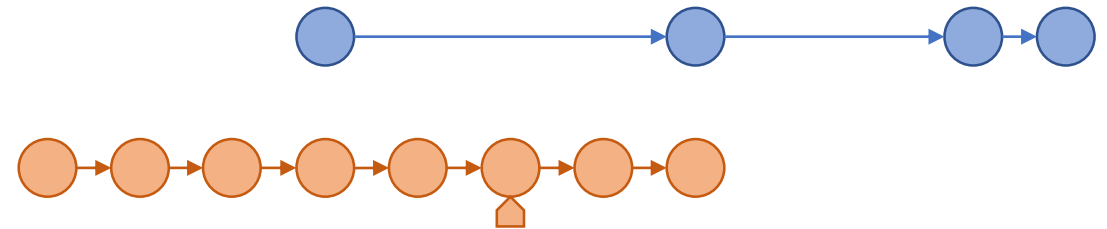
```
static int
visit_move(PyObject *op, void *arg)
{
    // ...
    if (gc_is_collecting(gc)) {
        gc_list_move(gc, tolist);
        gc_clear_collecting(gc);
    }
    // ...
}
```

```
static void
move_legacy_finalizer_reachable(PyGC_Head *finalizers)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        (void) Py_TYPE(FROM_GC(gc))->tp_traverse(FROM_GC(gc), visit_move, (void *)finalizers);
    }
}
```



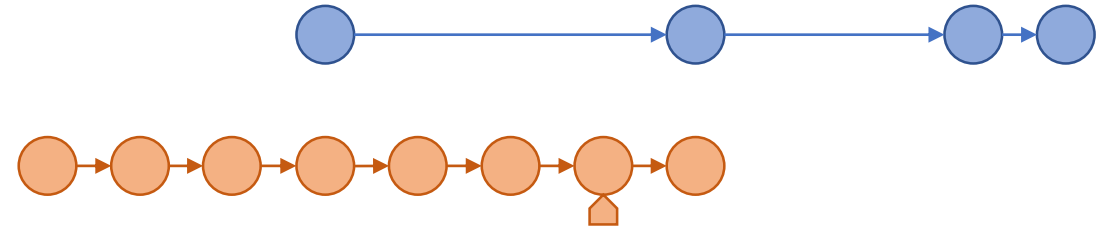
```
static int
visit_move(PyObject *op, void *arg)
{
    // ...
    if (gc_is_collecting(gc)) {
        gc_list_move(gc, tolist);
        gc_clear_collecting(gc);
    }
    // ...
}

static void
move_legacy_finalizer_reachable(PyGC_Head *finalizers)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        (void) Py_TYPE(FROM_GC(gc))->tp_traverse(FROM_GC(gc), visit_move, (void *)finalizers);
    }
}
```



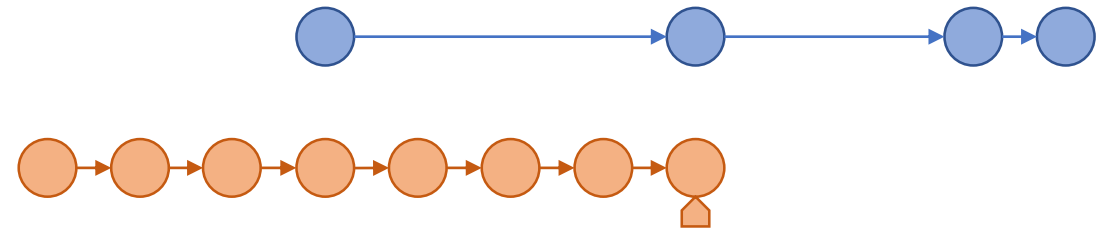
```
static int
visit_move(PyObject *op, void *arg)
{
    // ...
    if (gc_is_collecting(gc)) {
        gc_list_move(gc, tolist);
        gc_clear_collecting(gc);
    }
    // ...
}

static void
move_legacy_finalizer_reachable(PyGC_Head *finalizers)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        (void) Py_TYPE(FROM_GC(gc))->tp_traverse(FROM_GC(gc), visit_move, (void *)finalizers);
    }
}
```



```
static int
visit_move(PyObject *op, void *arg)
{
    // ...
    if (gc_is_collecting(gc)) {
        gc_list_move(gc, tolist);
        gc_clear_collecting(gc);
    }
    // ...
}

static void
move_legacy_finalizer_reachable(PyGC_Head *finalizers)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        (void) Py_TYPE(FROM_GC(gc))->tp_traverse(FROM_GC(gc), visit_move, (void *)finalizers);
    }
}
```



- Поиск недостижимых
- Определение несобираемых
- Обработка слабых ссылок
- Удаление объектов
- Определение воскрешенных объектов
- Очистка слабых ссылок
- Очистка циклов
- Перенос несобираемых

3.13

```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    PyGC_Head *gc;
    PyObject *op;          /* generally FROM_GC(gc) */
    PyWeakReference *wr;    /* generally a cast of op */
    PyGC_Head wrcb_to_call; /* weakrefs with callbacks to call */
    PyGC_Head *next;
    int num_freed = 0;

    gc_list_init(&wrcb_to_call);
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        PyWeakReference **wrlist;

        op = FROM_GC(gc);
        next = GC_NEXT(gc);

        if (PyWeakref_Check(op)) {
            _PyWeakref_ClearRef((PyWeakReference *)op);
        }

        if (! _PyType_SUPPORTS_WEAKREFS(Py_TYPE(op))) {
            continue;
        }

        wrlist = _PyObject_GET_WEAKREFS_LISTPTR_FROM_OFFSET(op);
        for (wr = *wrlist; wr != NULL; wr = *wrlist) {
            PyGC_Head *wrasgc;          /* AS_GC(wr) */

            _PyWeakref_ClearRef(wr);
            if (wr->wr_callback == NULL) {
                continue;
            }
            if (gc_is_collecting(AS_GC((PyObject *)wr))) {
                continue;
            }
            Py_INCREF(wr);

            wrasgc = AS_GC((PyObject *)wr);
            gc_list_move(wrasgc, &wrcb_to_call);
        }
    }

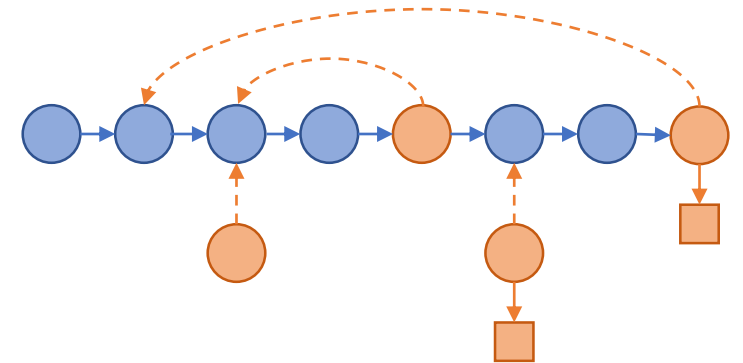
    while (! gc_list_is_empty(&wrcb_to_call)) {
        PyObject *temp;
        PyObject *callback;

        gc = (PyGC_Head*)wrcb_to_call._gc_next;
        op = FROM_GC(gc);
        wr = (PyWeakReference *)op;
        callback = wr->wr_callback;

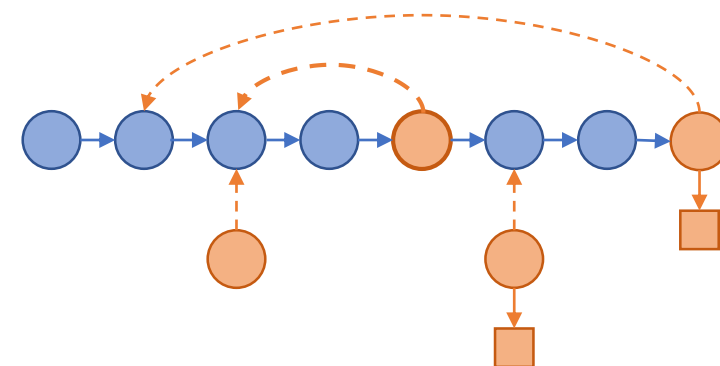
        temp = PyObject_CallOneArg(callback, (PyObject *)wr);
        if (temp == NULL) {
            PyErr_WriteUnraisable(callback);
        }
        else {
            Py_DECREF(temp);
        }

        Py_DECREF(op);
        if (wrcb_to_call._gc_next == (uintptr_t)gc) {
            gc_list_move(gc, old);
        }
        else {
            ++num_freed;
        }
    }

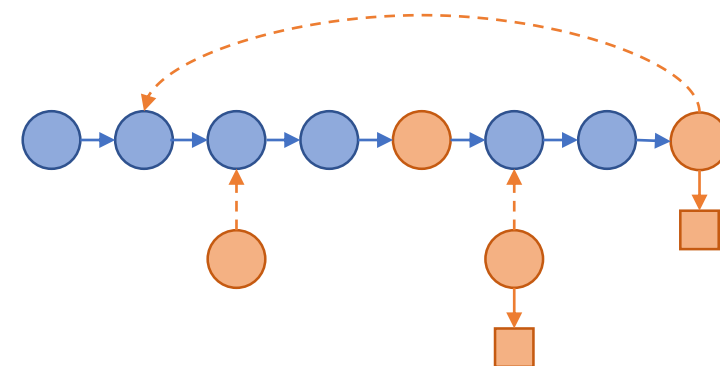
    return num_freed;
}
```



```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        // ...
        if (PyWeakref_Check(op)) {
            _PyWeakref_ClearRef((PyWeakReference *)op);
        }
        // ...
    }
    // ...
}
```



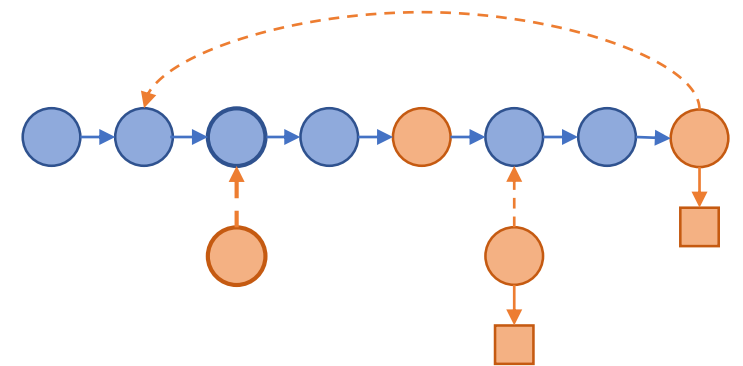
```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        // ...
        if (PyWeakref_Check(op)) {
            _PyWeakref_ClearRef((PyWeakReference *)op);
        }
        // ...
    }
    // ...
}
```



```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        // ...
        wrlist = _PyObject_GET_WEAKREFS_LISTPTR_FROM_OFFSET(op);
        for (wr = *wrlist; wr != NULL; wr = *wrlist) {
            PyGC_Head *wrasgc;

            _PyWeakref_ClearRef(wr);
            if (wr->wr_callback == NULL) {
                continue;
            }
            if (gc_is_collecting(AS_GC((PyObject *)wr))) {
                continue;
            }
            Py_INCREF(wr);

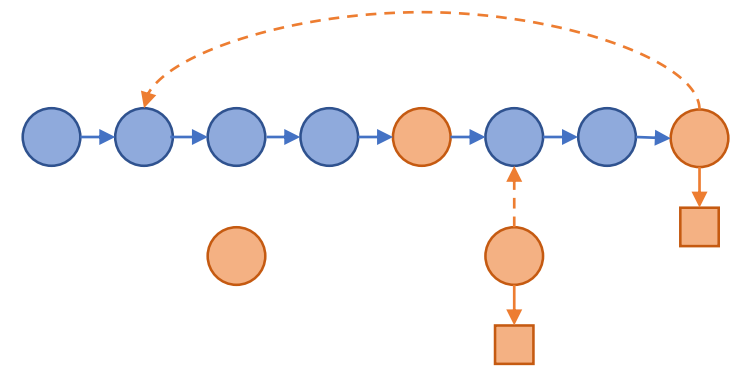
            wrasgc = AS_GC((PyObject *)wr);
            gc_list_move(wrasgc, &wrcb_to_call);
        }
        // ...
    }
}
```



```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        // ...
        wrlist = _PyObject_GET_WEAKREFS_LISTPTR_FROM_OFFSET(op);
        for (wr = *wrlist; wr != NULL; wr = *wrlist) {
            PyGC_Head *wrasgc;

            _PyWeakref_ClearRef(wr);
            if (wr->wr_callback == NULL) {
                continue;
            }
            if (gc_is_collecting(AS_GC((PyObject *)wr))) {
                continue;
            }
            Py_INCREF(wr);

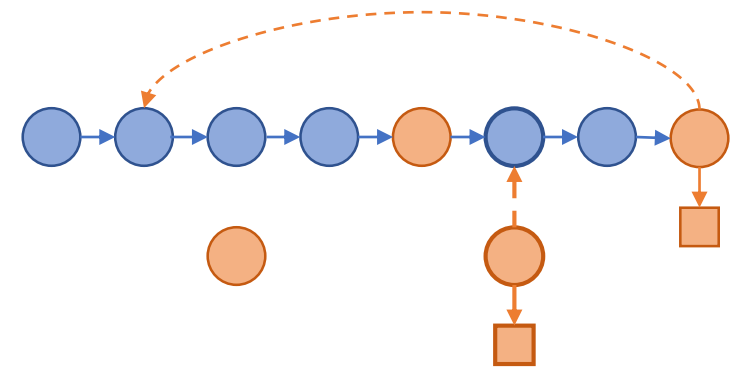
            wrasgc = AS_GC((PyObject *)wr);
            gc_list_move(wrasgc, &wrcb_to_call);
        }
        // ...
    }
}
```



```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    gc_list_init(&wrcb_to_call);
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        // ...
        wrlist = _PyObject_GET_WEAKREFS_LISTPTR_FROM_OFFSET(op);
        for (wr = *wrlist; wr != NULL; wr = *wrlist) {
            PyGC_Head *wrasgc;

            _PyWeakref_ClearRef(wr);
            if (wr->wr_callback == NULL) {
                continue;
            }
            Py_INCREF(wr);

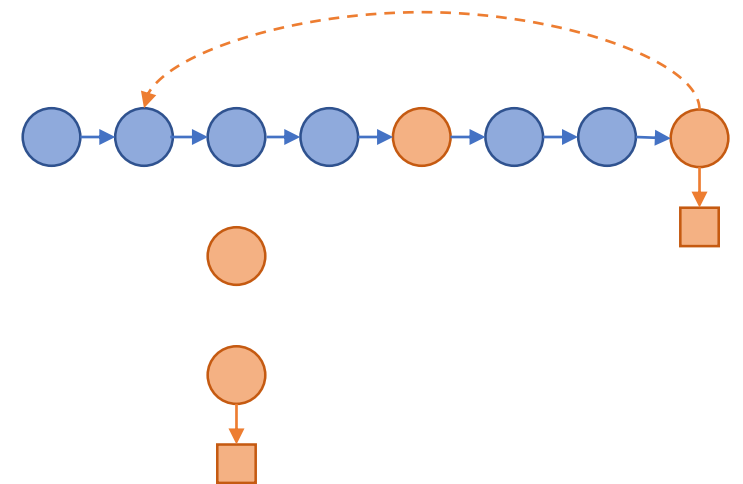
            wrasgc = AS_GC((PyObject *)wr);
            gc_list_move(wrasgc, &wrcb_to_call);
        }
    }
    // ...
}
```




```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    gc_list_init(&wrcb_to_call);
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        // ...
        wrlist = _PyObject_GET_WEAKREFS_LISTPTR_FROM_OFFSET(op);
        for (wr = *wrlist; wr != NULL; wr = *wrlist) {
            PyGC_Head *wrasgc;

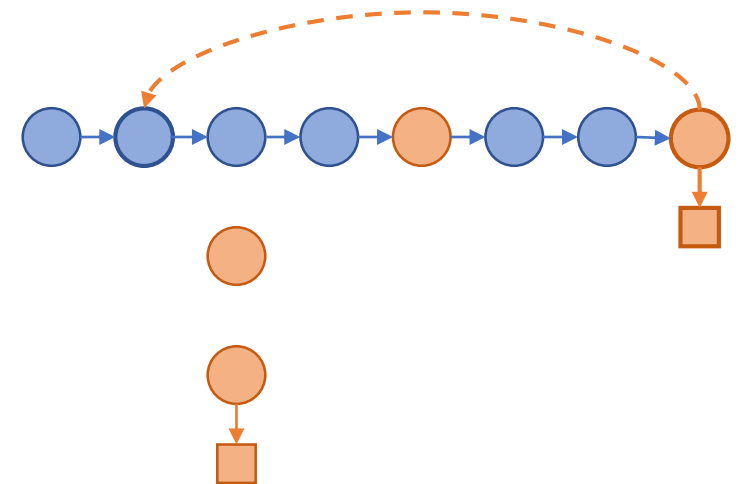
            _PyWeakref_ClearRef(wr);
            if (wr->wr_callback == NULL) {
                continue;
            }
            Py_INCREF(wr);

            wrasgc = AS_GC((PyObject *)wr);
            gc_list_move(wrasgc, &wrcb_to_call);
        }
    }
    // ...
}
```



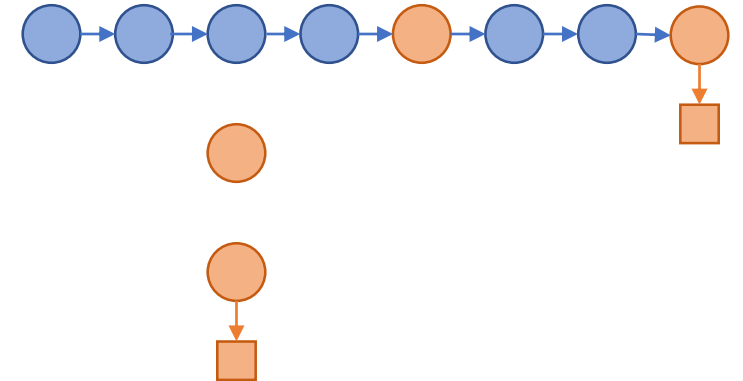
```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        // ...
        wrlist = _PyObject_GET_WEAKREFS_LISTPTR_FROM_OFFSET(op);
        for (wr = *wrlist; wr != NULL; wr = *wrlist) {
            PyGC_Head *wrasgc;

            _PyWeakref_ClearRef(wr);
            if (wr->wr_callback == NULL) {
                continue;
            }
            if (gc_is_collecting(AS_GC((PyObject *)wr))) {
                continue;
            }
            // ...
        }
    }
    // ...
}
```



```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        // ...
        wrlist = _PyObject_GET_WEAKREFS_LISTPTR_FROM_OFFSET(op);
        for (wr = *wrlist; wr != NULL; wr = *wrlist) {
            PyGC_Head *wrasgc;

            _PyWeakref_ClearRef(wr);
            if (wr->wr_callback == NULL) {
                continue;
            }
            if (gc_is_collecting(AS_GC((PyObject *)wr))) {
                continue;
            }
            // ...
        }
    }
    // ...
}
```



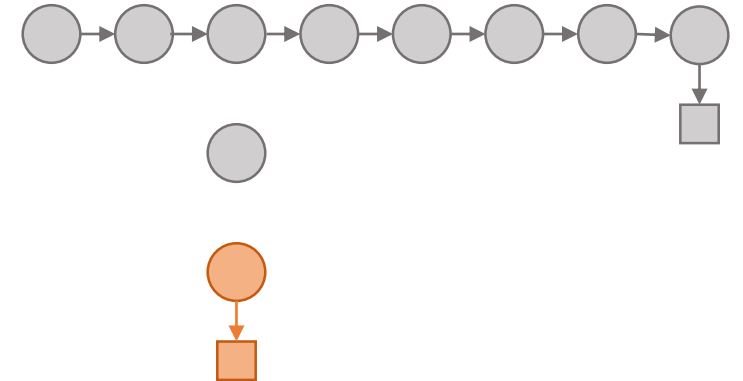
```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    while (! gc_list_is_empty(&wrcb_to_call)) {
        PyObject *temp;
        PyObject *callback;

        gc = (PyGC_Head*)wrcb_to_call._gc_next;
        op = FROM_GC(gc);
        wr = (PyWeakReference *)op;
        callback = wr->wr_callback;

        temp = PyObject_CallOneArg(callback, (PyObject *)wr);
        // ...

        Py_DECREF(op);
        if (wrcb_to_call._gc_next == (uintptr_t)gc) {
            gc_list_move(gc, old);
        }
        else {
            ++num_freed;
        }
    }

    return num_freed;
}
```



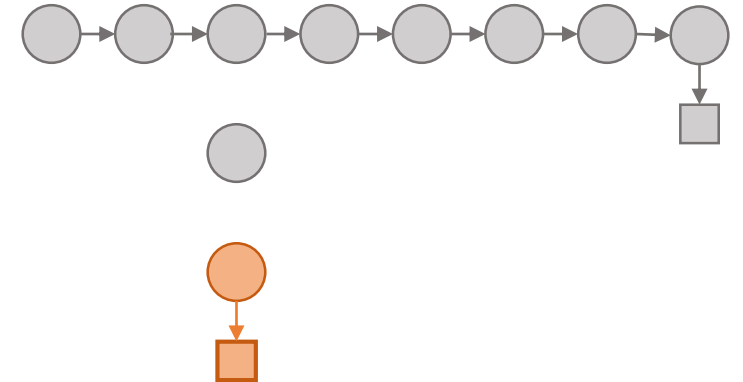
```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    while (! gc_list_is_empty(&wrcb_to_call)) {
        PyObject *temp;
        PyObject *callback;

        gc = (PyGC_Head*)wrcb_to_call._gc_next;
        op = FROM_GC(gc);
        wr = (PyWeakReference *)op;
        callback = wr->wr_callback;

        temp = PyObject_CallOneArg(callback, (PyObject *)wr);
        // ...

        Py_DECREF(op);
        if (wrcb_to_call._gc_next == (uintptr_t)gc) {
            gc_list_move(gc, old);
        }
        else {
            ++num_freed;
        }
    }

    return num_freed;
}
```

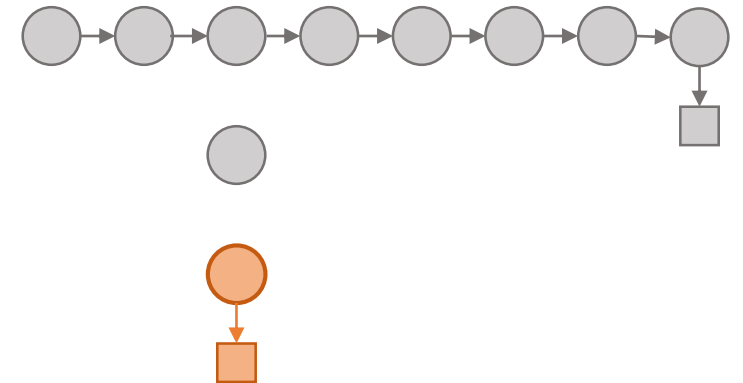


```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        // ...
        for (wr = *wrlist; wr != NULL; wr = *wrlist) {
            // ...
            Py_INCREF(wr);

            wrasgc = AS_GC((PyObject *)wr);
            gc_list_move(wrasgc, &wrcb_to_call);
        }
    }

    while (! gc_list_is_empty(&wrcb_to_call)) {
        // ...
        temp = PyObject_CallOneArg(callback, (PyObject *)wr);
        // ...
        Py_DECREF(op);
        // ...
    }

    return num_freed;
}
```



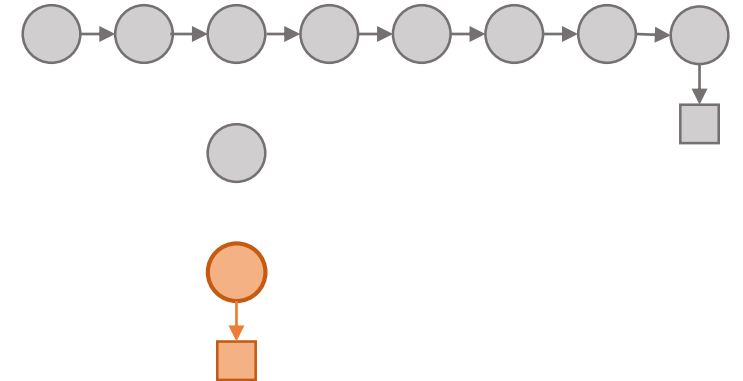
```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    while (! gc_list_is_empty(&wrcb_to_call)) {
        PyObject *temp;
        PyObject *callback;

        gc = (PyGC_Head*)wrcb_to_call._gc_next;
        op = FROM_GC(gc);
        wr = (PyWeakReference *)op;
        callback = wr->wr_callback;

        temp = PyObject_CallOneArg(callback, (PyObject *)wr);
        // ...

        Py_DECREF(op);
        if (wrcb_to_call._gc_next == (uintptr_t)gc) {
            gc_list_move(gc, old);
        }
        else {
            ++num_freed;
        }
    }

    return num_freed;
}
```



3.14


```
import gc
import weakref

class LateFin:
    __slots__ = ('ref',)

    def __del__(self):
        global func
        func = self.ref()

class Cyclic(tuple):
    __slots__ = ()

    def __del__(self):
        self[1].ref = weakref.ref(self[0])
        global latefin
        del latefin

latefin = LateFin()
def func():
    print('test')
cyc = tuple.__new__(Cyclic, (func, latefin))
func.__module__ = cyc

del func, cyc
gc.collect()

func()
```

```
import gc
import weakref

class LateFin:
    __slots__ = ('ref',)

    def __del__(self):
        global func
        func = self.ref()

class Cyclic(tuple):
    __slots__ = ()

    def __del__(self):
        self[1].ref = weakref.ref(self[0])
        global latefin
        del latefin

latefin = LateFin()
def func():
    print('test')
cyc = tuple.__new__(Cyclic, (func, latefin))
func.__module__ = cyc

del func, cyc
gc.collect()

func()
```

```
import gc
import weakref

class LateFin:
    __slots__ = ('ref',)

    def __del__(self):
        global func
        func = self.ref()

class Cyclic(tuple):
    __slots__ = ()

    def __del__(self):
        self[1].ref = weakref.ref(self[0])
        global latefin
        del latefin

latefin = LateFin()
def func():
    print('test')
cyc = tuple.__new__(Cyclic, (func, latefin))
func.__module__ = cyc

del func, cyc
gc.collect()

func()
```

```
import gc
import weakref

class LateFin:
    __slots__ = ('ref',)

    def __del__(self):
        global func
        func = self.ref()

class Cyclic(tuple):
    __slots__ = ()

    def __del__(self):
        self[1].ref = weakref.ref(self[0])
        global latefin
        del latefin

latefin = LateFin()
def func():
    print('test')
cyc = tuple.__new__(Cyclic, (func, latefin))
func.__module__ = cyc

del func, cyc
gc.collect()

func()
```

```
import gc
import weakref

class LateFin:
    __slots__ = ('ref',)

    def __del__(self):
        global func
        func = self.ref()

class Cyclic(tuple):
    __slots__ = ()

    def __del__(self):
        self[1].ref = weakref.ref(self[0])
        global latefin
        del latefin

latefin = LateFin()
def func():
    print('test')
cyc = tuple.__new__(Cyclic, (func, latefin))
func.__module__ = cyc

del func, cyc
gc.collect()

func()
```

```
import gc
import weakref

class LateFin:
    __slots__ = ('ref',)

    def __del__(self):
        global func
        func = self.ref()

class Cyclic(tuple):
    __slots__ = ()

    def __del__(self):
        self[1].ref = weakref.ref(self[0])
        global latefin
        del latefin

latefin = LateFin()
def func():
    print('test')
cyc = tuple.__new__(Cyclic, (func, latefin))
func.__module__ = cyc

del func, cyc
gc.collect()

func()
```

```
import gc
import weakref

class LateFin:
    __slots__ = ('ref',)

    def __del__(self):
        global func
        func = self.ref()

class Cyclic(tuple):
    __slots__ = ()

    def __del__(self):
        self[1].ref = weakref.ref(self[0])
        global latefin
        del latefin

latefin = LateFin()
def func():
    print('test')
cyc = tuple.__new__(Cyclic, (func, latefin))
func.__module__ = cyc

del func, cyc
gc.collect()

func()
```

```
import gc
import weakref
```

```
class LateFin:
    __slots__ = ('ref',)

    def __del__(self):
        global func
        func = self.ref()
```

```
class Cyclic(tuple):
    __slots__ = ()

    def __del__(self):
        self[1].ref()
        global latefin
        del latefin
```

Windows fatal exception: access violation

Current thread 0x0000ade0 (most recent call first):
File "b91636.py", line 39 in func
File "b91636.py", line 57 in <module>

```
latefin = LateFin()
def func():
    print('test')
cyc = tuple.__new__(Cyclic, (func, latefin))
func.__module__ = cyc
```

```
del func, cyc
gc.collect()
```

```
func()
```



```

873 + handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old, bool allow_callbacks)
874 {
875     PyGC_Head *gc;
876     PyObject *op;          /* generally FROM_GC(gc) */
877
878     PyGC_Head *next;
879     int num_freed = 0;
880
881 + if (allow_callbacks) {
882 +     gc_list_init(&wrcb_to_call);
883 + }
884
885     /* Clear all weakrefs to the objects in unreachable. If such a weakref
886     * also has a callback, move it into `wrcb_to_call` if the callback
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940 +
941 +     if (!allow_callbacks) {
942 +         continue;
943 +     }
944 +
945     if (wr->wr_callback == NULL) {
946         /* no callback */
947         continue;
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997 +
998 +     if (!allow_callbacks) {
999 +         return 0;
1000 +     }
1001
1002     /* Invoke the callbacks we decided to honor. It's safe to invoke them
1003     * because they can't reference unreachable objects.
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751 +
1752 +     stats->collected += handle_weakrefs(&unreachable, to, true);
1753 +     gc_list_validate_space(to, gcstate->visited_space);
1754 +     validate_list(to, collecting_clear_unreachable_clear);
1755 +     validate_list(&unreachable, collecting_set_unreachable_clear);
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799
2800
2801
2802
2803
2804
2805
2806
2807
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820
2821
2822
2823
2824
2825
2826
2827
2828
2829
2830
2831
2832
2833
2834
2835
2836
2837
2838
2839
2840
2841
2842
2843
2844
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860
2861
2862
2863
2864
2865
2866
2867
2868
2869
2870
2871
2872
2873
2874
2875
2876
2877
2878
2879
2880
2881
2882
2883
2884
2885
2886
2887
2888
2889
2890
2891
2892
2893
2894
2895
2896
2897
2898
2899
2900
2901
2902
2903
2904
2905
2906
2907
2908
2909
2910
2911
2912
2913
2914
2915
2916
2917
2918
2919
2920
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
2944
2945
2946
2947
2948
2949
2950
2951
2952
2953
2954
2955
2956
2957
2958
2959
2960
2961
2962
2963
2964
2965
2966
2967
2968
2969
2970
2971
2972
2973
2974
2975
2976
2977
2978
2979
2980
2981
2982
2983
2984
2985
2986
2987
2988
2989
2990
2991
2992
2993
2994
2995
2996
2997
2998
2999
3000
3001
3002
3003
3004
3005
3006
3007
3008
3009
3010
3011
3012
3013
3014
3015
3016
3017
3018
3019
3020
3021
3022
3023
3024
3025
3026
3027
3028
3029
3030
3031
3032
3033
3034
3035
3036
3037
3038
3039
3040
3041
3042
3043
3044
3045
3046
3047
3048
3049
3050
3051
3052
3053
3054
3055
3056
3057
3058
3059
3060
3061
3062
3063
3064
3065
3066
3067
3068
3069
3070
3071
3072
3073
3074
3075
3076
3077
3078
3079
3080
3081
3082
3083
3084
3085
3086
3087
3088
3089
3090
3091
3092
3093
3094
3095
3096
3097
3098
3099
3100
3101
3102
3103
3104
3105
3106
3107
3108
3109
3110
3111
3112
3113
3114
3115
3116
3117
3118
3119
3120
3121
3122
3123
3124
3125
3126
3127
3128
3129
3130
3131
3132
3133
3134
3135
3136
3137
3138
3139
3140
3141
3142
3143
3144
3145
3146
3147
3148
3149
3150
3151
3152
3153
3154
3155
3156
3157
3158
3159
3160
3161
3162
3163
3164
3165
3166
3167
3168
3169
3170
3171
3172
3173
3174
3175
3176
3177
3178
3179
3180
3181
3182
3183
3184
3185
3186
3187
3188
3189
3190
3191
3192
3193
3194
3195
3196
3197
3198
3199
3200
3201
3202
3203
3204
3205
3206
3207
3208
3209
3210
3211
3212
3213
3214
3215
3216
3217
3218
3219
3220
3221
3222
3223
3224
3225
3226
3227
3228
3229
3230
3231
3232
3233
3234
3235
3236
3237
3238
3239
3240
3241
3242
3243
3244
3245
3246
3247
3248
3249
3250
3251
3252
3253
3254
3255
3256
3257
3258
3259
3260
3261
3262
3263
3264
3265
3266
3267
3268
3269
3270
3271
3272
3273
3274
3275
3276
3277
3278
3279
3280
3281
3282
3283
3284
3285
3286
3287
3288
3289
3290
3291
3292
3293
3294
3295
3296
3297
3298
3299
3300
3
```

3.15

```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next

class Node(BaseNode):
    pass

BaseNode.next = Node()
BaseNode.next.next = Node()
```

```
Disassembly of <code object __del__ at 0x0...680, line 2>:
2          RESUME                0

3          LOAD_GLOBAL           0 (BaseNode)
          LOAD_ATTR              2 (next)
          LOAD_ATTR              2 (next)
          LOAD_GLOBAL           0 (BaseNode)
          STORE_ATTR             1 (next)
          LOAD_CONST            0 (None)
          RETURN_VALUE
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

2	RESUME	0
3	LOAD_GLOBAL	0 (BaseNode)
	LOAD_ATTR	2 (next)
	LOAD_ATTR	2 (next)
	LOAD_GLOBAL	0 (BaseNode)
	STORE_ATTR	1 (next)
	LOAD_CONST	0 (None)
	RETURN_VALUE	

```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next
```

```
class Node(BaseNode):
    pass
```

```
BaseNode.next = Node()
BaseNode.next.next = Node()
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

2	RESUME	0
3	LOAD_GLOBAL	0 (BaseNode)
	LOAD_ATTR	2 (next)
	LOAD_ATTR	2 (next)
	LOAD_GLOBAL	0 (BaseNode)
	STORE_ATTR	1 (next)
	LOAD_CONST	0 (None)
	RETURN_VALUE	

```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next
```

```
class Node(BaseNode):
    pass
```

```
BaseNode.next = Node()
BaseNode.next.next = Node()
```

```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next

class Node(BaseNode):
    pass

BaseNode.next = Node()
BaseNode.next.next = Node()
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

2	RESUME	0
3	LOAD_GLOBAL	0 (BaseNode)
	LOAD_ATTR	2 (next)
	LOAD_ATTR	2 (next)
	LOAD_GLOBAL	0 (BaseNode)
	STORE_ATTR	1 (next)
	LOAD_CONST	0 (None)
	RETURN_VALUE	

```
PyObject *
_PyObject_GenericGetAttrWithDict(PyObject *obj, PyObject *name,
                                  PyObject *dict, int suppress)
{
    // ...
    _PyType_LookupStackRefAndVersion(tp, name, &cref.ref);
    descr = PyStackRef_AsPyObjectBorrow(cref.ref);

    f = NULL;
    if (descr != NULL) {
        f = Py_TYPE(descr)->tp_descr_get;
        // ...
    }
    // ...
}
```

```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next

class Node(BaseNode):
    pass

BaseNode.next = Node()
BaseNode.next.next = Node()
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

2	RESUME	0
3	LOAD_GLOBAL	0 (BaseNode)
	LOAD_ATTR	2 (next)
	LOAD_ATTR	2 (next)
	LOAD_GLOBAL	0 (BaseNode)
	STORE_ATTR	1 (next)
	LOAD_CONST	0 (None)
	RETURN_VALUE	

```
unsigned int
_PyType_LookupStackRefAndVersion(PyTypeObject *type, PyObject
*name, _PyStackRef *out)
{
    // ...

    if (entry->version == type->tp_version_tag && entry->name ==
name) {
        *out = entry->value
            ? PyStackRef_FromPyObjectNew(entry->value)
            : PyStackRef_NULL;
        return entry->version;
    }

    // ...
}
```

```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next

class Node(BaseNode):
    pass

BaseNode.next = Node()
BaseNode.next.next = Node()
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

2	RESUME	0
3	LOAD_GLOBAL	0 (BaseNode)
	LOAD_ATTR	2 (next)
	LOAD_ATTR	2 (next)
	LOAD_GLOBAL	0 (BaseNode)
	STORE_ATTR	1 (next)
	LOAD_CONST	0 (None)
	RETURN_VALUE	

```
unsigned int
_PyType_LookupStackRefAndVersion(PyTypeObject *type, PyObject
*name, _PyStackRef *out)
{
    // ...

    if (entry->version == type->tp_version_tag && entry->name ==
name) {
        *out = entry->value
            ? PyStackRef_FromPyObjectNew(entry->value)
            : PyStackRef_NULL;
        return entry->version;
    }

    // ...
}
```



```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next

class Node(BaseNode):
    pass

BaseNode.next = Node()
BaseNode.next.next = Node()
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

2	RESUME	0
3	LOAD_GLOBAL	0 (BaseNode)
	LOAD_ATTR	2 (next)
	LOAD_ATTR	2 (next)
	LOAD_GLOBAL	0 (BaseNode)
	STORE_ATTR	1 (next)
	LOAD_CONST	0 (None)
	RETURN_VALUE	

```
PyObject *
_PyObject_GenericGetAttrWithDict(PyObject *obj, PyObject *name,
                                  PyObject *dict, int suppress)
{
    // ...
    _PyType_LookupStackRefAndVersion(tp, name, &cref.ref);
    descr = PyStackRef_AsPyObjectBorrow(cref.ref);

    f = NULL;
    if (descr != NULL) {
        f = Py_TYPE(descr)->tp_descr_get;
        // ...
    }
    // ...
}
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

```

2          RESUME          0

3          LOAD_GLOBAL     0 (BaseNode)
          LOAD_ATTR       2 (next)
          LOAD_ATTR       2 (next)
          LOAD_GLOBAL     0 (BaseNode)
          STORE_ATTR      1 (next)
          RESUME          0 (None)

```

```

class BaseNode:
    def __del__(self):
        BaseNode.next

```

```

class Node(BaseNode):
    pass

```

```

BaseNode.next = Node(
BaseNode.next.next =

```

Windows fatal exception: access violation

Current thread 0x000043b0 (most recent call first):

Garbage-collecting

File "b135552.py", line 3 in __del__

Current thread's C stack trace (most recent call first):

<cannot get C stack on this system>

```

PyObject *obj, PyObject *name,
PyObject *dict, int suppress)

```

```

_PyType_LookupStackRefAndVersion(tp, name, &cref.ref);
descr = PyStackRef_AsPyObjectBorrow(cref.ref);

```

```

f = NULL;
if (descr != NULL) {
    f = Py_TYPE(descr)->tp_descr_get;
    // ...
}
// ...

```

```

}

```

```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next

class Node(BaseNode):
    pass

BaseNode.next = Node()
BaseNode.next.next = Node()
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

2	RESUME	0
3	LOAD_GLOBAL	0 (BaseNode)
	LOAD_ATTR	2 (next)
	LOAD_ATTR	2 (next)
	LOAD_GLOBAL	0 (BaseNode)
	STORE_ATTR	1 (next)
	LOAD_CONST	0 (None)
	RETURN_VALUE	

```
static int
type_setattro(PyObject *self, PyObject *name, PyObject *value)
{
    // ...

    res = type_update_dict(type, dict, name, value, &old_value);

    // ...
}
```

```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next

class Node(BaseNode):
    pass

BaseNode.next = Node()
BaseNode.next.next = Node()
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

2	RESUME	0
3	LOAD_GLOBAL	0 (BaseNode)
	LOAD_ATTR	2 (next)
	LOAD_ATTR	2 (next)
	LOAD_GLOBAL	0 (BaseNode)
	STORE_ATTR	1 (next)
	LOAD_CONST	0 (None)
	RETURN_VALUE	

```
static int
type_update_dict(PyTypeObject *type, PyDictObject *dict, PyObject
*name, PyObject *value, PyObject **old_value)
{
    // ...

    type_modified_unlocked(type);

    // ...
}
```

```
class BaseNode:
    def __del__(self):
        BaseNode.next = BaseNode.next.next

class Node(BaseNode):
    pass

BaseNode.next = Node()
BaseNode.next.next = Node()
```

Disassembly of <code object __del__ at 0x0...680, line 2>:

2	RESUME	0
3	LOAD_GLOBAL	0 (BaseNode)
	LOAD_ATTR	2 (next)
	LOAD_ATTR	2 (next)
	LOAD_GLOBAL	0 (BaseNode)
	STORE_ATTR	1 (next)
	LOAD_CONST	0 (None)
	RETURN_VALUE	

```
static void
type_modified_unlocked(PyTypeObject *type)
{
    // ...
    PyObject *subclasses = lookup_tp_subclasses(type);
    if (subclasses != NULL) {
        // ...
        while (PyDict_Next(subclasses, &i, NULL, &ref)) {
            PyTypeObject *subclass = type_from_ref(ref);
            if (subclass == NULL) {
                continue;
            }
            // ...
        }
    }
    // ...
    set_version_unlocked(type, 0);
}
```

```
static int
handle_weakrefs(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    for (wr = *wrlist; wr != NULL; wr = *wrlist) {
        PyGC_Head *wrasgc;
        _PyWeakref_ClearRef(wr);
        if (wr->wr_callback == NULL) {
            continue;
        }
        if (gc_is_collecting(AS_GC((PyObject *)wr))) {
            continue;
        }
        Py_INCREF(wr);
        wrasgc = AS_GC((PyObject *)wr);
        gc_list_move(wrasgc, &wrcb_to_call);
    }
    // ...
}
```

```
static int
handle_weakref_callbacks(PyGC_Head *unreachable, PyGC_Head *old)
{
    // ...
    for (PyWeakReference *wr = *wrlist; wr != NULL; wr = next) {
        next = wr->wr_next;
        if (wr->wr_callback == NULL) {
            continue;
        }
        _PyWeakref_ClearRef(wr);
        if (gc_is_collecting(AS_GC((PyObject *)wr))) {
            continue;
        }
        Py_INCREF(wr);
        PyGC_Head *wrasgc = AS_GC((PyObject *)wr);
        gc_list_move(wrasgc, &wrcb_to_call);
    }
    // ...
}
```

- Поиск недостижимых
- Определение несобираемых
- Обработка слабых ссылок
- Удаление объектов
- Определение воскрешенных объектов
- Очистка слабых ссылок
- Очистка циклов
- Перенос несобираемых

```
static void
finalize_garbage(PyThreadState *tstate, PyGC_Head *collectable)
{
    destructor finalize;
    PyGC_Head seen;
    gc_list_init(&seen);

    while (!gc_list_is_empty(collectable)) {
        PyGC_Head *gc = GC_NEXT(collectable);
        PyObject *op = FROM_GC(gc);
        gc_list_move(gc, &seen);
        if (!_PyGC_FINALIZED(op) &&
            (finalize = Py_TYPE(op)->tp_finalize) != NULL)
        {
            _PyGC_SET_FINALIZED(op);
            Py_INCREF(op);
            finalize(op);
            Py_DECREF(op);
        }
    }
    gc_list_merge(&seen, collectable);
}
```



```
static void
finalize_garbage(PyThreadState *tstate, PyGC_Head *collectable)
{
    destructor finalize;
    PyGC_Head seen;
    gc_list_init(&seen);

    while (!gc_list_is_empty(collectable)) {
        PyGC_Head *gc = GC_NEXT(collectable);
        PyObject *op = FROM_GC(gc);
        gc_list_move(gc, &seen);
        if (!_PyGC_FINALIZED(op) &&
            (finalize = Py_TYPE(op)->tp_finalize) != NULL)
        {
            _PyGC_SET_FINALIZED(op);
            Py_INCREF(op);
            finalize(op);
            Py_DECREF(op);
        }
    }
    gc_list_merge(&seen, collectable);
}
```

```
static void
finalize_garbage(PyThreadState *tstate, PyGC_Head *collectable)
{
    destructor finalize;
    PyGC_Head seen;
    gc_list_init(&seen);

    while (!gc_list_is_empty(collectable)) {
        PyGC_Head *gc = GC_NEXT(collectable);
        PyObject *op = FROM_GC(gc);
        gc_list_move(gc, &seen);
        if (!_PyGC_FINALIZED(op) &&
            (finalize = Py_TYPE(op)->tp_finalize) != NULL)
        {
            _PyGC_SET_FINALIZED(op);
            Py_INCREF(op);
            finalize(op);
            Py_DECREF(op);
        }
    }
    gc_list_merge(&seen, collectable);
}
```

```
static void
finalize_garbage(PyThreadState *tstate, PyGC_Head *collectable)
{
    destructor finalize;
    PyGC_Head seen;
    gc_list_init(&seen);

    while (!gc_list_is_empty(collectable)) {
        PyGC_Head *gc = GC_NEXT(collectable);
        PyObject *op = FROM_GC(gc);
        gc_list_move(gc, &seen);
        if (!_PyGC_FINALIZED(op) &&
            (finalize = Py_TYPE(op)->tp_finalize) != NULL)
        {
            _PyGC_SET_FINALIZED(op);
            Py_INCREF(op);
            finalize(op);
            Py_DECREF(op);
        }
    }
    gc_list_merge(&seen, collectable);
}
```

- Поиск недостижимых
- Определение несобираемых
- Обработка слабых ссылок
- Удаление объектов
- Определение воскрешенных объектов
- Очистка слабых ссылок
- Очистка циклов
- Перенос несобираемых

```
static inline void
handle_resurrected_objects(PyGC_Head *unreachable, PyGC_Head* still_unreachable,
                           PyGC_Head *old_generation)
{
    gc_list_clear_collecting(unreachable);

    PyGC_Head* resurrected = unreachable;
    deduce_unreachable(resurrected, still_unreachable);
    clear_unreachable_mask(still_unreachable);

    gc_list_merge(resurrected, old_generation);
}
```

- Поиск недостижимых
- Определение несобираемых
- Обработка слабых ссылок
- Удаление объектов
- Определение воскрешенных объектов
- Очистка слабых ссылок
- Очистка циклов
- Перенос несобираемых

```
static void
clear_weakrefs(PyGC_Head *unreachable)
{
    PyGC_Head *gc;
    PyGC_Head *next;

    for (gc = GC_NEXT(unreachable); gc != unreachable; gc = next) {
        PyWeakReference **wrlist;

        PyObject *op = FROM_GC(gc);
        next = GC_NEXT(gc);

        if (PyWeakref_Check(op)) {
            _PyWeakref_ClearRef((PyWeakReference *)op);
        }

        if (! _PyType_SUPPORTS_WEAKREFS(Py_TYPE(op))) {
            continue;
        }

        wrlist = _PyObject_GET_WEAKREFS_LISTPTR_FROM_OFFSET(op);

        for (PyWeakReference *wr = *wrlist; wr != NULL; wr = *wrlist) {
            _PyWeakref_ClearRef(wr);
        }
    }
}
```

- Поиск недостижимых
- Определение несобираемых
- Обработка слабых ссылок
- Удаление объектов
- Определение воскрешенных объектов
- Очистка слабых ссылок
- Очистка циклов
- Перенос несобираемых


```
static void
delete_garbage(PyThreadState *tstate, GCState *gcstate,
               PyGC_Head *collectable, PyGC_Head *old)
{
    while (!gc_list_is_empty(collectable)) {
        PyGC_Head *gc = GC_NEXT(collectable);
        PyObject *op = FROM_GC(gc);

        if (gcstate->debug & _PyGC_DEBUG_SAVEALL) {
            if (PyList_Append(gcstate->garbage, op) < 0) {
                _PyErr_Clear(tstate);
            }
        }
        else {
            inquiry clear;
            if ((clear = Py_TYPE(op)->tp_clear) != NULL) {
                Py_INCREF(op);
                (void) clear(op);
                if (_PyErr_Occurred(tstate)) {
                    PyErr_FormatUnraisable("Exception ignored in tp_clear");
                }
                Py_DECREF(op);
            }
        }
        if (GC_NEXT(collectable) == gc) {
            gc_clear_collecting(gc);
            gc_list_move(gc, old);
        }
    }
}
```

```
static void
delete_garbage(PyThreadState *tstate, GCState *gcstate,
               PyGC_Head *collectable, PyGC_Head *old)
{
    while (!gc_list_is_empty(collectable)) {
        PyGC_Head *gc = GC_NEXT(collectable);
        PyObject *op = FROM_GC(gc);

        if (gcstate->debug & _PyGC_DEBUG_SAVEALL) {
            if (PyList_Append(gcstate->garbage, op) < 0) {
                _PyErr_Clear(tstate);
            }
        }
        else {
            inquiry clear;
            if ((clear = Py_TYPE(op)->tp_clear) != NULL) {
                Py_INCREF(op);
                (void) clear(op);
                if (_PyErr_Occurred(tstate)) {
                    PyErr_FormatUnraisable("Exception ignored in tp_clear");
                }
                Py_DECREF(op);
            }
        }
        if (GC_NEXT(collectable) == gc) {
            gc_clear_collecting(gc);
            gc_list_move(gc, old);
        }
    }
}
```

```
static void
delete_garbage(PyThreadState *tstate, GCState *gcstate,
               PyGC_Head *collectable, PyGC_Head *old)
{
    while (!gc_list_is_empty(collectable)) {
        PyGC_Head *gc = GC_NEXT(collectable);
        PyObject *op = FROM_GC(gc);

        if (gcstate->debug & _PyGC_DEBUG_SAVEALL) {
            if (PyList_Append(gcstate->garbage, op) < 0) {
                _PyErr_Clear(tstate);
            }
        }
        else {
            inquiry clear;
            if ((clear = Py_TYPE(op)->tp_clear) != NULL) {
                Py_INCREF(op);
                (void) clear(op);
                if (_PyErr_Occurred(tstate)) {
                    PyErr_FormatUnraisable("Exception ignored in tp_clear");
                }
                Py_DECREF(op);
            }
        }
        if (GC_NEXT(collectable) == gc) {
            gc_clear_collecting(gc);
            gc_list_move(gc, old);
        }
    }
}
```

```
static void
delete_garbage(PyThreadState *tstate, GCState *gcstate,
               PyGC_Head *collectable, PyGC_Head *old)
{
    while (!gc_list_is_empty(collectable)) {
        PyGC_Head *gc = GC_NEXT(collectable);
        PyObject *op = FROM_GC(gc);

        if (gcstate->debug & _PyGC_DEBUG_SAVEALL) {
            if (PyList_Append(gcstate->garbage, op) < 0) {
                _PyErr_Clear(tstate);
            }
        }
        else {
            inquiry clear;
            if ((clear = Py_TYPE(op)->tp_clear) != NULL) {
                Py_INCREF(op);
                (void) clear(op);
                if (_PyErr_Occurred(tstate)) {
                    PyErr_FormatUnraisable("Exception ignored in tp_clear");
                }
                Py_DECREF(op);
            }
        }
        if (GC_NEXT(collectable) == gc) {
            gc_clear_collecting(gc);
            gc_list_move(gc, old);
        }
    }
}
```

- Поиск недостижимых
- Определение несобираемых
- Обработка слабых ссылок
- Удаление объектов
- Определение воскрешенных объектов
- Очистка слабых ссылок
- Очистка циклов
- Перенос несобираемых

```
static void
handle_legacy_finalizers(PyThreadState *tstate,
                        GCState *gcstate,
                        PyGC_Head *finalizers, PyGC_Head *old)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        PyObject *op = FROM_GC(gc);

        if ((gcstate->debug & _PyGC_DEBUG_SAVEALL) || has_legacy_finalizer(op)) {
            if (PyList_Append(gcstate->garbage, op) < 0) {
                _PyErr_Clear(tstate);
                break;
            }
        }
    }

    gc_list_merge(finalizers, old);
}
```

```
static void
handle_legacy_finalizers(PyThreadState *tstate,
                        GCState *gcstate,
                        PyGC_Head *finalizers, PyGC_Head *old)
{
    PyGC_Head *gc = GC_NEXT(finalizers);
    for (; gc != finalizers; gc = GC_NEXT(gc)) {
        PyObject *op = FROM_GC(gc);

        if ((gcstate->debug & _PyGC_DEBUG_SAVEALL) || has_legacy_finalizer(op)) {
            if (PyList_Append(gcstate->garbage, op) < 0) {
                _PyErr_Clear(tstate);
                break;
            }
        }
    }

    gc_list_merge(finalizers, old);
}
```



РОСНЕФТЬ

СПАСИБО ЗА ВНИМАНИЕ!